A unified toolkit for advanced arithmetic in FHE*

Lorenzo Rovida

Polytechnic University of Turin, Italy
lorenzo.rovida@polito.it

Abstract. We suggest an extension of discrete–CKKS that enables advanced computations over real, Boolean and large integers (e.g., 64/128-bit), including standard arithmetic operations, comparison, shift, bit length, quotient, square root, and other non linear functions based on binary representation. The flexibility of our approach lies in the synergy between the integer and the real arithmetic layers. At the core of our approach lies indeed a novel non-iterative binary decomposition technique extracting the i-th bit of x by evaluating an approximation of $\lfloor x/2^i \rfloor \bmod 2$, very efficient when x has at most 8 bits.
We show how such procedure can decompose not only a value x, but also a more general $g(x)$ at no additional cost by generalizing the evaluation to $\lfloor g(x)/2^i \rfloor$, enabling fast LUT evaluation over integers. Interestingly, the output of our LUTs can be arbitrarily large at the cost of requiring redundant slots: this unlocks the computation of large seeds for Newton–Raphson algorithms, enabling for the first time the practical evaluation of (exact) non linear functions over large integers such as square root and quotient within the discrete–CKKS family of schemes.
By supporting computations over different domains, our framework is usable in various applications: smart contracts naturally fit our framework, as it efficiently supports both arbitrary logical gates and operations over large integers. It also allows integer computations on the output class of a real-valued neural network, such as metadata lookup.
Compared to the state-of-the-art by Gao and Zheng (CRYPTO '26), our solution does not require ad-hoc encoding or expensive integer to Boolean conversions, making it more suitable when some non arithmetic structure is involved. Our current limitation lies in the throughput, which is lower. We additionally provide an open source proof of concept for GPU, showing that our work has better latency and throughput than TFHE-rs on most arithmetic and non-arithmetic operations.

1 Introduction

Fully homomorphic encryption (FHE) has emerged as a very promising technique for computations over encrypted data since its introduction [Gen09]. Today we

* This is an updated version of the ePrint 2026/450 originally named "*A flexible and polynomial framework for integer arithmetic in CKKS* "

have a wide range of schemes that can be used to execute programs over ciphertexts according to a given use case; these mainly differ by their plaintext spaces (or domains). Most of them share the same core lattice-based construction based on the (Ring-)LWE [Reg09, LPR13a] trapdoor, albeit the way data is encoded and handled dramatically changes the available arithmetic.

New schemes, old functionalities. In the last years, new approaches have emerged to improve the performance of traditional FHE schemes. Among them, for example, we refer to [CLPX18, DMPS23, CCKS23, GV25, BFG+26, Kim25a, BK25, PZZ26, CPL26, GZ26]. Nonetheless, in spite of its "old" age, the FHEW or DM/CGGI approach [DM15, CGGI16] is still the one surrounded by the richest ecosystem of available operations, mostly implemented in the TFHE-rs library [Zam22]. However, its major limitation is that a ciphertext can encrypt up to a single n-bit integer. This is not competitive with respect to the single instruction, multiple data (SIMD) schemes such as BGV [BGV12], BFV [Bra12, FV12] or CKKS [CKKS17], that allow one to pack many elements in a single ciphertext.

A recent line of works, started in [DMPS23], tried to tackle the following question: can a discretized version of CKKS be competitive with FHEW/TFHE, at least in amortized time? Recent literature in what is now called *discrete-CKKS* [BCKS24, BKSS24, BK25, Kim25a, Kim25b, CPL26, GZ26] showed that the latter can be used to perform additions and multiplications in some subset of $\mathbb{Z}$ — even faster than in TFHE, for large word sizes — with thousands of elements in parallel, but failed in providing a similar framework for the set of all possible operations enabled by TFHE.

1.1 Our contribution

In this paper, we propose a framework that enables Boolean, real and integer computations with word sizes that are powers of two[1]. Previous instantiations of discrete–CKKS usually focus on a subset of operations on integers, making it harder to perform other operations or to switch to other domains. In general, previous works are insufficient for a more general-purpose encrypted computing device. This is the problem in, e.g., [ZYZ+24], also observed in [Kim25a, Section 4.2]. We therefore try to answer the following:

Research question. *Can we build an efficient CKKS-like scheme that supports Boolean, integer, and real operations without heavy conversions, covering a suite of logical and arithmetic functionalities such as $\sqrt{x}$ or x/y over large integers?*

We answer positively, in particular providing the following contributions:

1. We propose a framework enabling efficient switching between real, Boolean and integer computations inside CKKS. Since integers are decomposed in binary form, integer and Boolean operations share the same encoding. Real-valued circuits can be evaluated in leveled mode with 15 available levels (before or after the integer/Boolean circuit).

[1] Special primes can plausibly be adapted using hardware-inspired solutions

2. We introduce a novel method to binary decompose an integer $x \in [0, 2^b)$ in a *single step* by evaluating b polynomials (currently, with $b \leq 8$), each approximating $\lfloor x/2^i \rfloor \bmod 2$, with $i < b$, in parallel. This improves upon prior methods (e.g., [DAP+26]) by up to 8× fewer bootstrappings.
3. We generalize the decomposition procedure to decompose $\lfloor g(x)/2^i \rfloor \bmod 2$ for some $g(x) : \mathbb{Z}_{2^b} \to \mathbb{Z}_{2^k}$, to support single-step b-to-k look-up table (LUT) evaluations in the binary regime. Thanks to our radix-2 representation, k can be arbitrary since each polynomial computes one bit, differently from previous works [AKP25, DAP+26, CJKR26b], where only b-to-b LUTs are considered.
4. We use this procedure to compute the initial seed of integer Newton–Raphson algorithms via a 8-to-n LUT. This enables the practical evaluation of quotients and square roots, operations unsupported in other discrete–CKKS schemes. We thus define a first baseline for such functions. Division using our approach has lower latency than in TFHE-rs [Zam22] by at least 3× on GPU.
5. We implement standard arithmetic (addition, multiplication, logical shifts, comparisons) using hardware algorithms adapted for CKKS: Kogge–Stone adders [KS73] for additions with carry and divide-and-conquer multiplier reducing 2^k-bit products to 4-bit base case. We show that on multiplications our method has comparable latency with respect to the state-of-the-art [GZ26].

Our scheme well fits circuits that need mixed domains or non arithmetic operations on a modest number of values, while [GZ26]-style schemes are preferable over mostly pure arithmetic on massive batches. We state that [GZ26] has still a better throughput, and that our goal is to work on a modest number of inputs at the time. Orthogonally to [GZ26], we sacrifice throughput to improve latency.

Competitiveness in GPU. Along with an open source CPU-based implementation, we also provide one based on CUDA. Both are based on OpenFHE [AB+22] and the GPU version uses the FIDESlib backend [ADVLG+25], customized to support all the described operations. We report some latency results obtained in a proof of concept via GPU in Table 1.

Further comparisons — namely throughput and operations per second — are given later in Table 8.

Usability in concrete use cases. When non-linear functions and/or logical operations are required — typical in smart contracts and on chain computations [ABC+25, ZEL+25] — the radix-based approach (with digits not being bits) in discrete-CKKS has poor performance as numbers should be binary decomposed. Think as an example the case of [Pas26], where the Uniswap V2 [AZR20] backrunning amount is computed homomorphically, requiring the computation of both a comparison and the square root over 128-bits values. Indeed, the one regime where, e.g., [Kim25b, CPL26, GZ26] are preferable, is when circuits with mostly basic arithmetic structure are involved. We illustrate the practical impact of our framework with a few toy representative instructions in Table 2.

Table 1: Latency of our implementation and TFHE-rs [Zam22], evaluated by running `bench_integer_gpu` over the `460fe6c` commit. Both have been evaluated on a NVIDIA L40S. All evaluations in Ours include the final bootstrapping

Operation	Method	32-bit	64-bit	128-bit
Add/Compare $(+, >)$	Ours	**54 ms**	**57 ms**	**61 ms**
	[Zam22]	117 ms (2.2×)	146 ms (2.6×)	204 ms (3.3×)
Multiplication $(\times)$	Ours	525 ms	**622 ms**	**732 ms**
	[Zam22]	**370 ms** (0.7×)	852 ms (1.3×)	2073 ms (2.8×)
Boolean $(\wedge, \vee, \oplus)$	Ours	**16 ms**	**16 ms**	**16 ms**
	[Zam22]	29 ms (1.8×)	29 ms (1.8×)	29 ms (1.8×)
Quotient $(/)$	Ours	**2.7 s**	**4.4 s**	**6.7 s**
	[Zam22]	8.5 s (3.1×)	14.0 s (3.2×)	36.8 s (5.5×)

1.2 Related works

Our proposal lies in the set of works that enable integer arithmetic over CKKS. We now review some of them, in chronological order.

- Boneh and Kim [BK25] first presented an elegant FHE scheme able to perform computations over very large integers modulo a prime by constructing a nested RNS system. However, their (lazy) modular reduction approach combined with RNS does not guarantee exactness with respect to the target modulus, making non-arithmetic operations (e.g., comparisons) difficult.
- Kim [Kim25b] adopted a particular encoding (*coefficients in slots* [JPK+24, Section 3]) to speed up integer multiplications as polynomial multiplications. An approximate digit carry for arithmetic operations is then used, reducing the bootstrapping cost to $O(\log k)$. However, in order to perform non–arithmetic operations, a unique digit representation must be restored, hence requiring again $O(k)$ bootstrappings.
- Cha, Park and Lee [CPL26] improved the previous construction by allowing to compute an exact representation requiring $O(\log(k))$ bootstrappings by introducing a so-called lightweight carry propagation. Such representation allows one to perform some non-arithmetic operations, such as logical shifts by multiples of 8 and comparisons.
- Gao and Zheng [GZ26] propose a very interesting solution to perform arithmetic by using a specific encoding from which it is possible to switch to Boolean (and vice-versa). This solution is the closest to ours, as they deal with integer and Boolean arithmetic in the same construction. Unfortunately, switching between the two representations seems hard. For instance, switching from integers to Boolean appears to be roughly four times slower than an integer multiplication [GZ26, Table 2, A2B column].

Table 2: Latency comparison between [GZ26] and our framework on some 64-bit instructions on a single value. Non arithmetic operations in [GZ26] require a conversion to Boolean. Assuming $a \geq c > b \geq d$. Computed on an M5 Max CPU

Description	Instruction	Ours, CPU	[GZ26], CPU	Ours, CUDA
In smart contracts				
Vote eligibility	age $> 18 \wedge \overline{\text{voted}}$	**2.5 s** (1×)	52 s (21×)	58 ms (0.02×)
Split money	total $\gg 2$	**0.1 s** (1×)	53 s (530×)	< 1 ms
Amount check	$a > 0 \wedge a \leq$ balance	**5.0 s** (1×)	104 s (21×)	116 ms (0.02×)
Uniswap V2 mint	$\sqrt{a \cdot b}$	**221 s** (1×)	*Not available*	5.8 s (0.02×)
Generic operations				
Normalization	$\lfloor (a-b)/(c-b) \rfloor$	**158 s** (1×)	*Not available*	5.5 s (0.03×)
Distance	$\sqrt{(a-b)^2 + (c-d)^2}$	**244 s** (1×)	*Not available*	6.8 s (0.02×)
Basic arithmetic	$(a \cdot b)^2 + c$	33 s (1×)	**25 s** (0.74×)	1.39 s (0.04×)

By reviewing these works, it seems like a strong limitation is given by the encoding that one is willing to use. Moreover, while achieving impressive results in additions and multiplications, these discrete–CKKS schemes typically fail in enabling the evaluation of efficient mixed domain circuits, non arithmetic and non linear operations; whereas, on the other hand, the DM/CGGI community built a massive set of operations in this sense, making it still preferable for arbitrary circuits composed of cross domain operations such as comparisons, quotients, square roots, LUTs, and so on. It is thus not surprising that encrypted Ethereum virtual machine environments are usually based on DM/CGGI [ABC+25]. Our goal is to build a framework based on binary representation, as it is perhaps the most flexible and every operation (arithmetic and not) can be performed there. We complete the comparison with the current literature in Appendix A.

1.3 Technical overview

Binary decomposition using polynomials. A very interesting capability of CKKS is the evaluation of *different* polynomials in *different* slots of a ciphertext. By taking advantage of it, we show how to binary decompose an integer $x \in \mathbb{Z}_{2^b}$, repeated b times, in a single step. Specifically, we evaluate b polynomials in parallel, each interpolating the function $\lfloor x/2^i \rfloor \bmod 2$ for $0 \leq i < b$. We provide an intuition of this approach in Figure 1. Since computations in CKKS are SIMD, this operation roughly costs the same as the evaluation of a single polynomial. This will be useful for us as, for example, we will perform binary multiplication using a recursive algorithm that will have as the base case a 4-bit multiplier, based on the standard CKKS multiplication, whose result is then binary decomposed. Note that, while we will not store large numbers directly, this procedure will be useful for a number of operations we will perform.

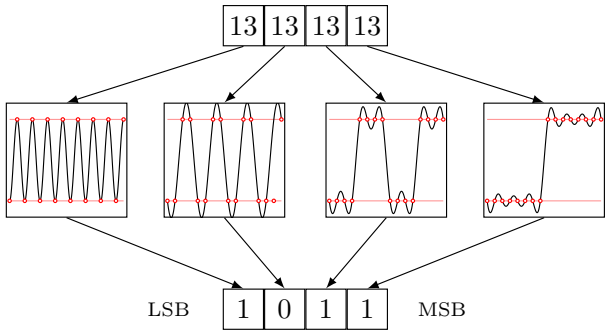


Fig. 1: Decomposing $x \in [0, 2^b)$ by evaluating b polynomials, with $x = 13, b = 4$.

LUT evaluation. The one above was our first intuition. However, we also observed that, from a functional bootstrapping perspective [CJP21, BKSS24, AKP25], what we do in the decomposition is simply the evaluation of an identity map $x \mapsto x$ to transform some value x into its binary form. Hence, we want to generalize such mapping to $x \mapsto g(x)$ for any $g(x) : \mathbb{Z}_{2^b} \to \mathbb{Z}_{2^k}$, for any integer k. It turns out we can simply generalize the approximated function to $\lfloor g(x)/2^i \rfloor \bmod 2$. The main trick lies in how SIMD is used: in our case we encode *redundant* values to parallelize different operations. In other words, we use redundancy to speed up computations. For instance we can evaluate an AES S-box, followed by a cleaning bootstrapping, in roughly 5 seconds. In [AKP25, Table 1], this takes around 47 seconds under a similar setup. To give an intuition, one simply replaces the polynomials in Figure 1 with polynomial computing the bits of $g(x)$.

Combining LUTs and Newton–Raphson. A typical strategy to compute functions such as square roots is by combining LUTs and root finding algorithms to refine the result. For instance, the `RSQRTPS` instruction in x86 CPUs [Int] computes the first bits of $1/\sqrt{x}$ which are typically refined by iterative algorithms. We take this path and show how to do something similar in our scheme with quotients and square roots.

Simple arithmetic operations. As our large integers are represented in bits, we implement arithmetic operation following standard algorithms from hardware literature. To reduce the multiplicative depth of additions, we use the Kogge–Stone adder (KSA) [KS73] as it has logarithmic depth. This also enables comparisons (i.e., $x > y$) by checking the carry out bit of $x + \hat{y}$ (with $\hat{y} = 1 - y$). Equality checks can be performed as well in a very efficient way. In our construction, every binary number is followed by some zeros to improve multiplication efficiency. Hence, a logical shift ($x \ll s$) can be simply implemented as an automorphism (i.e., a rotation) and has negligible complexity compared to the other operations. For multiplications, we take advantage of the practical

binary decomposition and use, as the core of our general 2^k multiplier, a 4-bit multiplier. In general, for 2^k arithmetic, we use a standard divide-and-conquer approach. The two operands $\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^k$ are split into high bits $\boldsymbol{x}_h, \boldsymbol{y}_h$ and low bits $\boldsymbol{x}_\ell, \boldsymbol{y}_\ell$; this reduces the large n-bit multiplication to four smaller $n/2$-bit multiplications.

2 Preliminaries

We use lowercase to indicate scalars as a and bold lowercase to indicate vectors as $\boldsymbol{a} := (a_0, \dots a_{n-1})$. Let $\mathsf{Bits}(a) : \mathbb{Z}_{2^b} \to \{0,1\}^b$ be the bit decomposition function and $\mathsf{Pow}(\boldsymbol{x}) : \{0,1\}^b \to \mathbb{Z}$ be its inverse (binary encoding). If not specified, we assume logarithms in base two, i.e., $\log := \log_2$. Let $N > 1$ be some power of two and $Q > 1$ an integer modulus. Let $\mathcal{R} := \mathbb{Z}[X]/(X^N + 1)$ be the quotient ring of polynomials modulo the $2N$-th cyclotomic polynomial $\Phi_{2N}(x) = X^N + 1$. Also define $\mathcal{R}_Q := \mathcal{R}/Q\mathcal{R}$. The CKKS scheme is based on a particular type of encoding, called *slots encoding*, in which a discrete Fourier transform (DFT) is used to enable SIMD operations. Let $\mathsf{DFT} : \mathbb{R}[X]/(X^N + 1) \to \mathbb{C}^{N/2}$ be a DFT defined by evaluating a polynomial $p(X)$ in $\left(\xi^{5^i}\right)_{0 \le i < N/2}$ with ξ being a complex primitive $2N$-root of unity and $\mathsf{iDFT} : \mathbb{C}^{N/2} \to \mathbb{R}[X]/(X^N + 1)$ be its inverse. When using the "multiplicative levels" term, we refer to the standard CKKS multiplication consuming one modulus of the chain.

2.1 CKKS in a nutshell

Key generation in the CKKS scheme follows from RLWE, so we set the public key and secret key to be:

$$\mathsf{pk} := (\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e}, \boldsymbol{a}) \in (\mathbb{Z}_Q[X]/(X^N + 1))^2, \quad \mathsf{sk} := \boldsymbol{s} \in \mathbb{Z}_Q[X]/(X^N + 1),$$

for uniformly random $\boldsymbol{a} \in \mathcal{R}_Q$ and small $\boldsymbol{s}, \boldsymbol{e} \in \mathcal{R}_Q$. Encryption is performed as in RLWE schemes, although encoding in CKKS follows these procedures:

1. Given some *cleartext* $\boldsymbol{m} \in \mathbb{C}^{N/2}$, evaluate $\mathsf{iDFT}(\boldsymbol{m})$ to put the message into the so-called *slots-encoding*.
2. The obtained *plaintext* $\mathsf{iDFT}(\boldsymbol{m})$ is scaled by a large factor $\Delta > 0$, this controls the precision of the encoding.
3. Since the polynomial now lives in $\mathbb{R}[X]/(X^N + 1)$, apply some rounding operation ([LPR13b, Section 2.4.2]) to make it compatible with RLWE.

We therefore obtain the following functions:

$$\mathsf{Encode}(\boldsymbol{m}) := \lfloor \Delta \cdot \mathsf{iDFT}(\boldsymbol{m}) \rceil \in \mathcal{R}, \quad \mathsf{Decode}(\boldsymbol{e}) := \frac{1}{\Delta} \cdot \mathsf{DFT}\,(\boldsymbol{e}) \in \mathbb{C}^{N/2} \quad (1)$$

This type of (approximate) encoding enables SIMD operations, since point-wise addition or multiplication of plaintext slots corresponds to polynomial addition (or multiplication, respectively) in the iDFT domain via the homomorphisms:

$$\mathsf{DFT}(\mathsf{iDFT}(\boldsymbol{a}) + \mathsf{iDFT}(\boldsymbol{b})) = \boldsymbol{a} + \boldsymbol{b} \quad \text{and} \quad \mathsf{DFT}(\mathsf{iDFT}(\boldsymbol{a}) \cdot \mathsf{iDFT}(\boldsymbol{b})) = \boldsymbol{a} \odot \boldsymbol{b}.$$

We remark that, in practice, we use the RNS-CKKS instantiation described in [KPP22]. We refer to Appendix E.1 for more information about CKKS and its bootstrapping operation.

Discrete-CKKS. A recent line of works, called *discrete*-CKKS [DMPS23], proposes to restrict the cleartext of CKKS from $\mathbb{C}$ to a subset of it, e.g., $\mathbb{Z}_p \subset \mathbb{C}$ for a small p. This gives several advantages over exact schemes, as it allows one to compute over small integers but with a large throughput and to evaluate real polynomials over them. However, ring structure is typically simulated [KN25].

Definition 1 (CKKS variants). *We define CKKS over different domains:* $\mathcal{R}$-CKKS*,* $\mathcal{I}$-CKKS*, and* $\mathcal{B}$-CKKS *are restrictions of the CKKS scheme in which plaintexts encode an approximate vector* $\boldsymbol{v}' = \boldsymbol{v} + \boldsymbol{e}$*, where:*

- *for* $\mathcal{R}$-CKKS*,* $v_i \in \mathbb{R}$ *and* $\boldsymbol{e}$ *is not distinguishable*
- *for* $\mathcal{I}$-CKKS*,* $v_i \in \mathbb{Z}$ *and we require* $|e_i| < 0.5$ *for correct decryption*
- *for* $\mathcal{B}$-CKKS*,* $v_i \in \{0, 1\}$ *and we require* $|e_i| < 0.5$ *for correct decryption*

For $\mathcal{I}/\mathcal{B}$-CKKS*, the value of* $\max(\boldsymbol{e})$ *is called the* error *of the ciphertext.*

Clearly, the decoding procedure changes according to the current CKKS mode, we provide for completeness the different decoding algorithms for each type of ciphertext in Appendix E.3. Also note that we will use $\mathcal{B}$-CKKS to both represent large numbers (using a radix-2 decomposition) and Boolean vectors. Conversely, the $\mathcal{I}$-CKKS representation is only used as an intermediate form *during* other procedures, in particular as input to polynomials.

2.2 A digression on polynomials

Throughout this paper we will mention different types of polynomials, we thus briefly introduce them below, although we refer the interested reader to the beautiful book by Trefethen [Tre19].

Chebyshev polynomials. Chebyshev polynomials are a family of orthogonal polynomials defined, without loss of generality, on the $[-1, 1]$ interval, with the key property that they minimize the maximum approximation error. They satisfy the recurrence $T_0 = 1$, $T_1 = x$, and $T_{n+1}(x) = 2x\,T_n(x) - T_{n-1}(x)$.

In the context of CKKS, Chebyshev polynomials are the standard tool for approximating non-linear functions. Their popularity is given by two key points: (i) it is easy to find a set of coefficients such that the resulting polynomial is *logarithmically far* from being the best [Tre19, Theorem 16.1] and (ii) there are efficient algorithms [PS73, CCS19] to evaluate a degree d polynomial using $O(\log(d))$ multiplicative levels.

Trigonometric polynomials. In general, the discrete Fourier transform (DFT) takes a finite set of equidistant samples of a function and represents them as a finite linear combination of complex exponentials. If the samples arise from a function defined on an interval $[0, 2\rho]$, the corresponding frequencies are integer multiples of π/ρ. For real-valued data, this representation can equivalently be written as a *trigonometric polynomial*.

Definition 2 (Trigonometric polynomial). *Let $n \in \mathbb{Z}^+$. A 2ρ-periodic* trigonometric polynomial *of degree n is a function of the form*

$$p(x) = \frac{a_0}{2} + \sum_{k=1}^{n} \left(a_k \cos\left(\frac{k\pi x}{\rho}\right) + b_k \sin\left(\frac{k\pi x}{\rho}\right)\right) \quad \text{with } a_k, b_k \in \mathbb{R}$$

In particular, given a set of equidistant points we can define a unique trigonometric polynomial, as proved by the following.

Theorem 1 ([SB02, Theorem 2.3.1.5], adapted). *For any support points $(x_i, f(x_i))$, with $0 \leq i < n$, f real and $x_i = 2i\pi/n$ (equidistant), there exists a unique trigonometric polynomial $p(x)$ with*

$$p(x_i) = f(x_i) \quad \text{for } 0 \leq i < n.$$

In [AKP25, Section A.3], the authors give a nice overview on the evaluations of trigonometric polynomials outside of the bootstrapping, in particular observing that natively evaluating these polynomial (as in [CKKL24]) might be expensive.

Hermite interpolation. Hermite interpolation extends ordinary polynomial interpolation by matching not just function values at given nodes, but also derivatives up to a specified order. Declined in the context of discrete-CKKS, this means that one can (i) define a set of constraint points over which the polynomial passes through, (ii) set the derivatives equal to zero in such points and (iii) fit a polynomial using Hermite interpolation. This allows one to (iv) evaluate a LUT over some encrypted input (typically integer, or appropriately discretized) and at the same time to reduce the CKKS noise. Alexandru, Kim and Polyakov [AKP25] initiated the (explicit) study of such polynomials in trigonometric form to ease their computation under CKKS (following some nice ideas in [BCKS24, BKSS24]). They noted that such case is known in trigonometric interpolation theory as a special case of the first-order trigonometric Hermite interpolation [SV65] (refer to [AKP25, Theorem 1]). They thus provide a way to compute such interpolation as a polynomial in base $e^{2\pi ix}$ [AKP25, Corollary 1]. In practice, during the bootstrapping procedure, instead of simulating a modular reduction modulo p [CHK+18], they use a Chebyshev interpolation to approximate the complex exponential $m_i \approx e^{2\pi i m_i/p}$, which additionally performs the modular reduction (this step is called EvalExp). After that, evaluating a LUT involves the evaluation of a Hermite interpolation (in a step named EvalLUT).

3 Methodology

We represent values in three encoding modes: $\mathcal{R}$-CKKS ($\mathbb{R}$), $\mathcal{I}$-CKKS ($\mathbb{Z}_{2^k}$) and $\mathcal{B}$-CKKS ($\{0,1\}^b$). We stress that we will represent large integers in $\mathcal{B}$-CKKS, but the $\mathcal{I}$-CKKS representation will be very useful for multiplications and LUTs. We provide an illustration in Figure 2, along with arrows describing their transformations, which are discussed below — note that, from now on, we use LSB-first representation. The main packing bottleneck is given by $\mathcal{B}$-

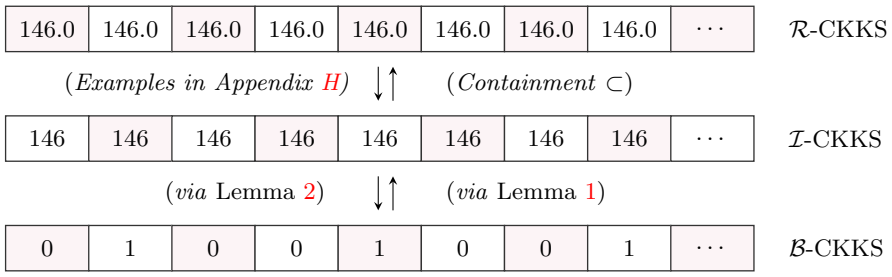


Fig. 2: How data types are stored, and how to switch between them

CKKS, which requires multiple binary slots to represent a single integer. Also observe that the $\mathcal{R}$-to-$\mathcal{I}$ transformation does not really make sense in general, although we report an example of a possible transformation: given a real vector, compute its integer indexing or the Argmax; practical transformations are given in Appendix H. This can be useful, e.g., to work over the predicted class of some NN output (which can be computed in $\mathcal{R}$-CKKS), or to make an encrypted NN more resilient to model extraction attacks by only revealing the output class (the so-called *hard-label* setting). We further observe that such a pipeline is hardly applicable in, e.g., [GZ26], as the CKKS modulus used there supports only about 20 multiplicative levels. In contrast, our CKKS modulus supports 29 multiplicative levels, making NNs evaluation much more practical.

3.1 Boolean to integer (or $\mathcal{B}$-to-$\mathcal{I}$)

Given a $\mathcal{B}$-CKKS ciphertext, it is easy to convert it into an $\mathcal{I}$-CKKS ciphertext, given that the CKKS parameters are sufficient to accommodate the (magnitude of the) converted value.

Lemma 1 ($\mathcal{B}$-to-$\mathcal{I}$ conversion). *There exists an efficient algorithm that, given a $\mathcal{B}$-CKKS ciphertext encoding some vector $\boldsymbol{x} \in \mathbb{Z}_{2^b}$, converts it into an $\mathcal{I}$-CKKS ciphertext requiring one multiplication,* $\log(b)$ *additions and rotations.*

Proof. Let $\boldsymbol{x} = (x_0, \ldots, x_{b-1}) \in \{0,1\}^b$ and compute the SIMD product with the first b powers of two as $\boldsymbol{y} = (x_0 2^0, \ldots, x_{b-1} 2^{b-1})$. A rotate and sum procedure, requiring $\log(b)$ rotations and additions, sums all entries of $\boldsymbol{y}$ into the first slot, yielding $y_0 = \sum_{i=0}^{b-1} x_i 2^i = \mathsf{Pow}(\boldsymbol{x})$., i.e., the integer value associated with $\boldsymbol{x}$. □

This transformation is the most straightforward and fast one. It may require an additional multiplication to mask slots containing partial results in case one wants to repeat the obtained integer along the other slots.

3.2 Integer to Boolean (or $\mathcal{I}$-to-$\mathcal{B}$)

In order to convert an $\mathcal{I}$-CKKS ciphertext into a $\mathcal{B}$-CKKS one, we need to perform a binary decomposition. In particular, current techniques (e.g., [AKP25, Appendix D] or [DAP+26, Section 3]) rely on digit extraction, where the least significant is iteratively extracted. However, the complexity of such approach is $O(\log_p(2^b))$ functional bootstrappings [DAP+26, Algorithm 2], where $2^b > p$ are the starting and the final moduli, respectively (in our case, $p = 2$ and $2^b = 256$). To binary decompose — for instance — a number in $[0, 256)$ in bits, this would require $\log_2(256) = 8$ functional bootstrappings.

Simpler and more direct decompositions. While in principle we could use standard techniques to convert an $\mathcal{I}$-CKKS ciphertext into a $\mathcal{B}$-CKKS one, we are required to evaluate this phase as fast as possible, as it will be used in many operations, such as integer multiplications and integer non-linear function evaluation. Therefore, we extend the set of available tools to perform a bit decomposition $\mathbb{Z}_{2^b} \to \{0,1\}^{\log(b)}$ of a number in $\mathbb{Z}_b$.

In particular, we now show that, when $b \leq 8$, we can efficiently decompose a value $x \in \mathbb{Z}_{2^b}$ in a single SIMD polynomial evaluation. Previous works extract one bit at a time, while our key observation is that the binary decomposition of some value in (some restriction of) $\mathbb{Z}$ can be evaluated by extracting each bit independently, using periodic functions over 0 and 1.

In particular, the i-th bit of some value $x \in \mathbb{Z}_{2^b}$, with $0 \leq i < b$, is equal to $e_i(x) := \lfloor x/2^i \rfloor \bmod 2$. Unfortunately, computing such function (which we will call *bit extraction function*) in CKKS is not easy because of its high non-linearity. Nonetheless, we are not required to evaluate $e_i(x)$ on a closed subset of $\mathbb{R}$, but rather to interpolate through the integer points (or some representatives) as e_i is only defined on values from a subset of $\mathbb{Z}$. We thus relax the previous argument and introduce a formal definition.

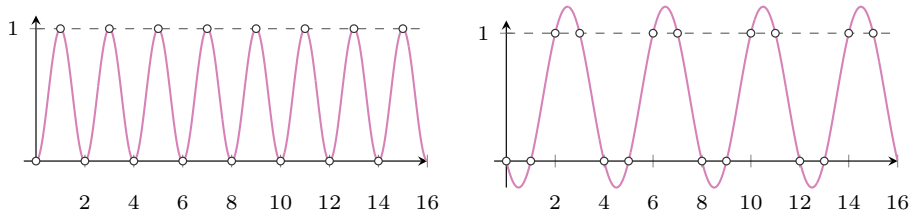
Definition 3 (i-th bit extraction, relaxed). *The i-th bit of some value $x \in \mathbb{Z}_{2^b}$ is equal to $f(x)$, where f is such that $f(x) = e_i(x)\ \forall\ x \in [0, 2^b) \subset \mathbb{N}$.*

Given that Theorem 1 enables to define a unique trigonometric polynomial that interpolates through all the integers in $[0, n]$, we can derive, as a consequence, that a trigonometric polynomial constructed from the equidistant integer samples of $e_k(x)$ is a bit extraction function by Definition 3.

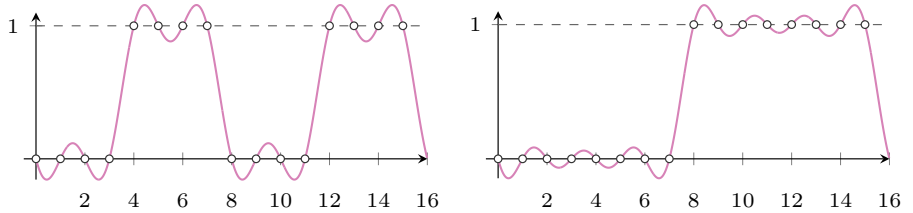
Corollary 1 (Existence of trigonometric bit extraction function). *There exists some $f(x)$ such that the unique trigonometric polynomial $p(x)$ interpolating $f(x)$ is $p(x) = e_i(x)\ \forall\ x \in [0, 2^b) \subset \mathbb{N}$.*

Proof. Given $e_i(x) = \lfloor x/2^i \rfloor \bmod 2$, set $n = 2^\rho$ for some $\rho \in \mathbb{N}$ with $\rho \geq b$, so that by Theorem 1 there exists a unique trigonometric polynomial $p(x)$ interpolating f at the nodes $x_j = 2j\pi/n$ for $j = 0, \ldots, n-1$. In particular, at each node $p(x_j) = f(j) = \lfloor j/2^i \rfloor \bmod 2 = e_i(j)$. □

We can therefore use, e.g., the DFT of $\lfloor x/2^i \rfloor \bmod 2$ — sampled over the integers in $[0, 2^b) \subset \mathbb{Z}$ — to build a trigonometric polynomial that enables the extraction of the i-th bit given some integer, with $0 \leq i < b$. In Figure 3 we illustrate, as an example, four trigonometric polynomials that can be used to decompose an integer in $\mathbb{Z}_{16}$. Although the target function is discontinuous, the



(a) $k = 0$ extracts the 1st bit (b) $k = 1$ extracts the 2nd bit



(c) $k = 2$ extracts the 3rd bit (d) $k = 3$ extracts the 4th bit

Fig. 3: Fourier expansions of $\lfloor x/2^i \rfloor \bmod 2$ for $i \in [0, 1, 2, 3]$ derived from a DFT, in the $[0, 16)$ interval. Notice that the same polynomials are used in Figure 1

smooth trigonometric polynomial becomes easier to approximate *directly*[2] with Chebyshev polynomials. This enables the construction of practical polynomial approximations for discrete bit extraction via Chebyshev polynomials.

The core limit of such conversion is that converting integer values $x \in [0, 2^b) \subset \mathbb{R}$ requires a polynomial approximation in the interval $[0, 2^b)$, whose size grows exponentially with b. Luckily, for our construction needs, $b = 8$ will be sufficient[3]. Note that in our proof of concept we used Chebyshev approximations of degree 451 — evaluable using 9 multiplicative levels — to obtain b

[2] We stress that we approximate the trigonometric polynomials with Chebyshev approximations, we do not go into trigonometric basis

[3] We note that our method is compatible with previous ones: one can, given e.g., a 10-bit value, extract the first 2 bits using digit extraction and the last 8 with our method. Extending our method to $b > 8$ is an improvement left for future work

polynomials $\hat{p}_i$ of precision $\|\hat{p}_i - p_i\|_\infty < 10^{-9}$ in $[0, 256)$. Finally, we can put together everything to define a conversion from $\mathcal{I}$-CKKS to $\mathcal{B}$-CKKS in Algorithm 1. Notice that the output of the algorithm is bootstrapped via binary bootstrapping (BinBTS), introduced in [BCKS24] and discussed in Appendix E.1. Intuitively, this bootstrapping squeezes the error from order e down to order e^2.

Algorithm 1 $\mathcal{I}$-to-$\mathcal{B}$, or binary decomposition via multiple polynomials

Input: $b \in \mathbb{N}$, an $\mathcal{I}$-CKKS ciphertext $\boldsymbol{c}$ encrypting $x \in [0, 2^b)$ with $c_0 = c_j$ for $j < b$
Output: A $\mathcal{B}$-CKKS ciphertext $\boldsymbol{b}$, encrypting $\mathsf{Bits}(x)$

1: *Offline/plain phase: obtaining Chebyshev coefficients*
2: **for** $i \leftarrow 0$ to $b - 1$
3: $\quad f_i(x) \leftarrow \lfloor x/2^i \rfloor \bmod 2$
4: $\quad p_i \leftarrow \mathrm{DFT}(f_i)$ over $[0, 2^b)$ ▷ *p_i computes the i-th bit of x*
5: $\quad \hat{p}_i \leftarrow$ Chebyshev approximation of p_i in $[0, 2^b)$
6: *Online/encrypted phase*
7: $b_i \leftarrow \hat{p}_i(c_i)$ ▷ *This is computed in a single* SIMD *call to Paterson-Stockmeyer*
8: **return** $\mathsf{BinBTS}(\boldsymbol{b})$ ▷ *The bootstrapped $\boldsymbol{b}$ is a $\mathcal{B}$-CKKS ciphertext*

Lemma 2 ($\mathcal{I}$-to-$\mathcal{B}$ correctness). *Ignoring the noise, Algorithm 1 is correct.*

Proof. Let $x \in [0, 2^b) \subset \mathbb{N}$ be the value encoded by every one of the input slots $c_0, \ldots, c_{b-1}$. By Definition 3, for every $0 \le i < b$ the i-th bit of x equals $\lfloor x/2^i \rfloor \bmod 2 = f_i(x)$, namely the function defined on line 3. So it suffices to show $b_i = f_i(x)$ for every i (with $\boldsymbol{b}$ being the output value). By Corollary 1, there exists a unique trigonometric polynomial p_i interpolating f_i exactly at the integers of $[0, 2^b)$, line 4 computes precisely this polynomial. Line 5 replaces p_i by a Chebyshev approximation $\hat{p}_i$ on $[0, 2^b)$, accurate enough (see the following paragraph) s.t. the result is accepted by $\mathcal{B}$-CKKS decryption. Lastly, correctness of BinBTS follows from [BCKS24, Theorem 1], which also decreases the error. □

Noise and composability of $\mathcal{B}$-to-$\mathcal{I}$ and $\mathcal{I}$-to-$\mathcal{B}$. We intentionally avoid discussing about the noise in the descriptions of algorithms, as it strongly depends on the fixed CKKS parameters. For the parametrization used in our experiments, empirical results (available in our public repository) show that given a $\mathcal{B}$-CKKS ciphertext with error in the order of 10^{-5}, the output of $\mathcal{B}$-to-$\mathcal{I}$ followed by $\mathcal{I}$-to-$\mathcal{B}$ using a polynomial of degree 451 and a $3x^2 - 2x^3$ cleaning polynomial [DMPS23] has error of order 10^{-7}. More aggressive parameters can relax the constraints, but we are not interested in that at this stage. We will later give empirical results on the noise of our main operations in Appendix D. Finding the smallest CKKS parameters that achieve correctness with overwhelming probability in the presence of noise is left as an open problem.

3.3 Generalizing the decomposition: look-up tables

We now claim that the previously presented approach is implicitly computing a LUT taking as input a $\mathcal{I}$-CKKS ciphertext and returning a $\mathcal{B}$-CKKS ciphertext. In particular, the i-th bit extraction function $\lfloor x/2^i \rfloor \bmod 2$ computes the decomposition of x. In other words, Algorithm 1 implicitly computes the $x \mapsto x$ LUT while decomposing x. Similarly to the generalizations done in the past [CJP21, AKP25] to the identity function evaluated during the bootstrapping, we generalize the decomposition function by observing that:

$$\lfloor x/2^i \rfloor \bmod 2 \quad \xrightarrow{\text{is a special case of}} \quad \lfloor g(x)/2^i \rfloor \bmod 2 \tag{2}$$

where $g(x) : \mathbb{Z}_{2^b} \to \mathbb{Z}_{2^k}$ is some function, with $k \in \mathbb{Z}$ being any integer. Notice that, in the case of Algorithm 1 we have that $g(x) = x$ is the identity function. By using (2), we can both (i) interpolate over some function $g(x)$ and (ii) obtain a binary decomposition of the result.

Notice that the input size of the LUT is limited by b, but the output having size k bits can be arbitrarily large. This is very useful for our non-linear function evaluation, or in cases such as [CJKR26a], where embeddings for NNs are computed server–side starting from an encrypted index. We thus have a way to generalize Algorithm 1 to arbitrary LUTs, and we do so in Algorithm 2.

Algorithm 2 b-to-k LUT evaluation

Input: An $\mathcal{I}$-CKKS ciphertext $\boldsymbol{c}$ in the shape $c_0 = c_j$ for all $0 \leq j < k$, with $c_0 < 2^b$.
Some $g(x) : \mathbb{Z}_{2^b} \to \mathbb{Z}_{2^k}$ over which the LUT is computed, for some $k \in \mathbb{Z}$
Output: A $\mathcal{B}$-CKKS ciphertext containing $\mathsf{Bits}(g(c_0))$

Offline/plain phase: obtaining Chebyshev coefficients
for $i \leftarrow 0$ to $k-1$
  $f_i(x) \leftarrow \lfloor g(x)/2^i \rfloor \bmod 2$
  $p_i \leftarrow \mathsf{DFT}(f_i)$ over $[0, 2^b)$ ▷ p_i *computes the* i*-th bit of* $g(x)$
  $\hat{p}_i \leftarrow$ Chebyshev approximation of p_i in $[0, 2^b)$
Online/encrypted phase
$b_i \leftarrow \hat{p}_i(c_i)$
return $\mathsf{BinBTS}(\boldsymbol{b})$

Corollary 2 (LUT evaluation correctness). *Ignoring the noise, Algorithm 2 is correct.*

The proof follows similarly to the one of Lemma 2. Notice that, since the $\mathcal{B}$-to-$\mathcal{I}$ transformation is almost trivial (Lemma 1), this can easily be extended as an $\mathcal{I}$-to-$\mathcal{I}$ LUT, with some use cases in the functional bootstrapping à la [AKP25].

Differences with current approaches. Hermite interpolation is the main building block of many recent works [AKP25, CJKR26b]. Our approach is somewhat similar, but with a different execution and tools by taking advantage of our

binary encoding. From a high-level point of view, we reduce the complexity of LUTs by computing each bit of the result separately. This, however, requires our encoding to be redundant, and thus our approach has lower throughput than, e.g., [AKP25]. This phase is then followed by BinBTS, where the modulus is recovered and the noise is reduced. The result is the same as in [AKP25], the cleaned result of a LUT, however the two executions are different. In general, we believe that our approach could be easily implementable as an FBT procedure using Hermite à la [AKP25]. The main caveat would be to evaluate a different LUT in the different slots. However, we conjecture this to be slower than our pipeline[4]. We position our LUT approach as between DM/CGGI and AKP-FBT in terms of both latency and throughput.

4 Standard integer arithmetic operations

In this section we provide some algorithmic definitions of the primitive operations enabled by our framework, including binary addition with carry ($x + y$), comparisons ($x \leq y$), logical shifts ($x \ll s$) and multiplications ($x \cdot y$). Note that, from now on, we always assume a power of two number of bits n.

Addition. As previously mentioned, additions are evaluated using the Kogge–Stone adder (KSA) [KS73], described in Algorithm 3. It turns out that this

Algorithm 3 Kogge–Stone adder (KSA)

Input: Two binary numbers $\boldsymbol{x} = (x_0, \ldots, x_{n-1}, 0)$, $\boldsymbol{y} = (y_0, \ldots, y_{n-1}, 0)$ followed by one zero
Output: Binary sum $\boldsymbol{s} = (s_0 \ldots s_{n-1})$ and carry-out c_n

Procedure $\mathsf{Add}(\boldsymbol{x}, \boldsymbol{y})$
  Define the propagate ($\boldsymbol{p}$) and generate ($\boldsymbol{g}$) vectors:
  $p_i \leftarrow x_i \oplus y_i, \quad g_i \leftarrow x_i \wedge y_i$
  Compute prefix carries using Kogge–Stone parallel prefix:
  for $d \leftarrow 0$ **to** $\log(n) - 1$
    $g_i \leftarrow g_i + (p_i \wedge g_{i-2^d})$
    $p_i \leftarrow p_i \wedge p_{i-2^d}$
  $c_i \leftarrow g_{i-1}, \quad c_0 \leftarrow 0$
  $s_i \leftarrow (x_i \oplus y_i) \oplus c_i$ ▷ *For* $0 \leq i \leq n$
  return $\mathsf{BinBTS}(\boldsymbol{s})$

algorithm well fits the SIMD schemes, as it enables the computation of most of the workload in parallel. It requires $\log(n) + 2$ multiplicative levels, thus $O(1)$

[4] An 8-bit LUT using a Hermite interpolation of order $n = 1$ takes roughly 47 seconds in [AKP25, Table 1], while in our case the LUT followed by BinBTS takes around 5 seconds. We remark that our BinBTS is possibly the easiest bootstrapping for CKKS

bootstrapping in our setup. However, it is important for correctness that the binary vectors are followed by (at least) one zero. An extensive description of the algorithm is in Appendix E.4.

Subtraction. Subtractions can be performed as $\boldsymbol{x} - \boldsymbol{y} = \boldsymbol{x} + \hat{\boldsymbol{y}} + 1$, where $\hat{y}_i = 1 - y_i$. We can use a slightly different version of Algorithm 3 taking as input also a carry-in set to one. Refer to Appendix E.5 for further discussions. The complexity is the same as additions.

Comparison. Comparisons as $x \geq y$ can be implemented similarly to subtractions. Given two n-bit numbers $\boldsymbol{x} = (x_0, \dots x_{n-1})$ and $\boldsymbol{y} = (y_0, \dots, y_{n-1})$, we first reverse the bits of $\boldsymbol{y}$ as $\hat{y}_i = 1 - y_i$, and then we add $\boldsymbol{x} + \hat{\boldsymbol{y}}$ using e.g., KSA. The result of the comparison will be the last carry present in the n-th position. In practice:

$$\boldsymbol{x} \geq \boldsymbol{y} \iff \text{carry out}(\boldsymbol{x} - \boldsymbol{y}) = 1. \tag{3}$$

Algorithm 4 Comparison $x \geq y$

Input: Two binary numbers $\boldsymbol{x} = (x_0, \dots, x_{n-1})$, $\boldsymbol{y} = (y_0, \dots, y_{n-1})$
Output: Comparison as $x \geq y$ in s_n

1: **Procedure** $\mathsf{Comp}(\boldsymbol{x}, \boldsymbol{y})$
2: $\quad \boldsymbol{s} \leftarrow \mathsf{Sub}(\boldsymbol{x}, \hat{\boldsymbol{y}})$ ▷ *Algorithm 11*
3: $\quad$ **return** s_n ▷ *Or return* $1 - s_n$ *if* $x < y$ *is required*

Equality. On the other hand, checking whether $x = y$ is slightly different. Since we are working with bits, we can follow a simple approach that is compute a XOR between two n-bit $\mathcal{B}$-CKKS ciphertext, sum the resulting n bits, and check whether the sum is equal to zero. In practice: $\boldsymbol{x} = \boldsymbol{y} \iff \sum_{i=0}^{n}(x_i \oplus y_i) = 0$. Checking whether the sum is zero can be done in a very simple way, using techniques from [RLB26], namely an approximation of the $\text{sinc}(x) := \sin(x\pi)/x\pi$ function over the $[0, n]$ interval (which has, additionally, cleaning properties [RLB26, Lemma 5]). The overall complexity is similar to the previous case (i.e., $O(1)$ bootstrapping). We formalize this operation in Algorithm 5.

Algorithm 5 Equality $a = b$

Input: Two n-bit numbers $\boldsymbol{x} = (x_0, \dots, x_{n-1})$, $\boldsymbol{y} = (y_0, \dots, y_{n-1})$
Output: Equality test, c_0 contains 1 if $\boldsymbol{x} = \boldsymbol{y}$ and 0 otherwise

1: **Procedure** $\mathsf{Eq}(\boldsymbol{x}, \boldsymbol{y})$
2: $\quad \boldsymbol{c} \leftarrow \boldsymbol{x} \oplus \boldsymbol{y}$
3: $\quad \boldsymbol{c} \leftarrow \mathsf{RotateAndSum}(\boldsymbol{c}, n)$ ▷ $c_0 \leftarrow \sum_{i=0}^{n-1} c_i$
4: $\quad$ **return** $\mathsf{BinBTS}(\text{sinc}(\boldsymbol{c}))_0$ ▷ *If the sum is zero return 1 otherwise return 0*

Multiplication. Our goal is to evaluate multiplications by taking advantage of SIMD computations, not to improve throughput, but to improve latency. For this reason we will require more than n slots to represent an n-bit number. We further elaborate below. Our reference construction is a divide-and-conquer multiplier as it is (i) highly parallelizable, which is good for SIMD computations and (ii) has logarithmic depth, which is good to reduce the number of bootstrappings[5]. As a first step, observe that we need to preprocess the inputs $\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^n$ by defining low bits $(\boldsymbol{x}_\ell, \boldsymbol{y}_\ell)$ and high bits $(\boldsymbol{x}_h, \boldsymbol{y}_h)$, as illustrated in Figure 4. Each operand is followed by zeroes required to accommodate the partial result,

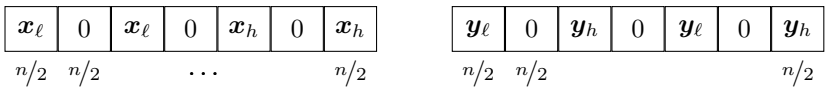


Fig. 4: Processing $\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^n$ before the multiplication.

observe indeed that each vector now requires $4n$ slots. We can now compute the actual multiplication as:

$$2^n \cdot \boldsymbol{x}_h \cdot \boldsymbol{y}_h + 2^{n/2} \cdot \boldsymbol{x}_h \cdot \boldsymbol{y}_\ell + 2^{n/2} \cdot \boldsymbol{x}_\ell \cdot \boldsymbol{y}_h + \boldsymbol{x}_\ell \cdot \boldsymbol{y}_\ell. \tag{4}$$

In particular, we use a 4-bit multiplier as the base case and evaluate all larger multiplications recursively. Power of two multiplications (logical shifts) are evaluated as simple rotations.

The base case $(n = 4)$. We implement the base case as follows. Given two values $\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^4$, we convert them to $\mathcal{I}$-CKKS ciphertexts (Lemma 1) and we perform the standard CKKS multiplication. The result will lie in $[0, 225] \subset \mathbb{Z}$ (modulo some small error), and we convert it back to $\mathcal{B}$-CKKS (Lemma 2). Notice that since the interval is smaller than $[0, 256)$, we can also use smaller polynomial degrees (in our proof of concept we will use 401, giving maximum error 10^{-7} in $[0, 225]$ for each of the eight bits).

The recursive case $(n > 4)$. Following the divide-and-conquer approach, we split the multiplication in four smaller multiplications based on the $n/2$-bit multiplier, until we reach the base case $n = 4$. Then, for each call, we require to evaluate four n-bit additions using the KSA. We can reduce them to a single one, after two rounds of carry-save adder (CSA), additional details are given in Appendix E.6. Since we want to parallelize every operation, we want to reduce all the base cases into a single ciphertext. To begin, we need to accommodate the four $n/2$-bit operands, plus the same amount of zeroes, we will thus start with the requirement of $4n$ slots. Subsequently, since we will recursively split each operand

[5] We intentionally avoid Karatsuba approach as it requires to perform more additions, which are the main bottleneck in our algorithm.

in high and low bits of size $n/4$, each of the four operands will recursively require four times half their space, for a total of twice the slots required for each recursive call, refer to Figure 5 for a visual intuition. This continues recursively, yielding

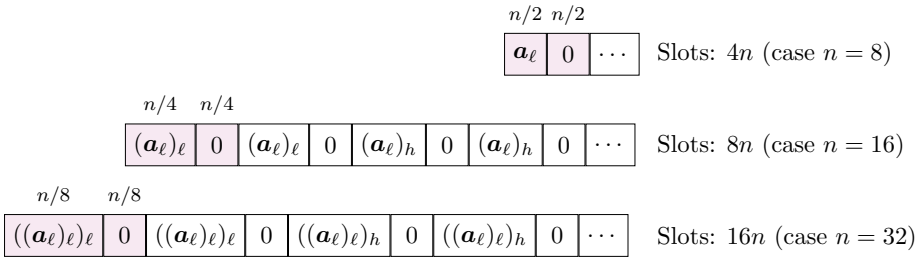


Fig. 5: The number of required slots to allow for parallelization doubles at each recursive call.

the following corollary.

Corollary 3 (Multiplication slots requirements). *An n-bit divide-and-conquer multiplication, with $n = 2^k$, requires $n^2/2$ slots to be executed with SIMD operations. Up to $2s/n^2$ values can be packed with s CKKS slots.*

Proof. Let $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^n$ be the two operands. At each recursive call we expand the size of the operands from n to $4n$ as:

$$\begin{aligned} \boldsymbol{a}' &:= (\boldsymbol{a}_\ell, \boldsymbol{0}, \boldsymbol{a}_\ell, \boldsymbol{0}, \boldsymbol{a}_h, \boldsymbol{0}, \boldsymbol{a}_h, \boldsymbol{0}) \in \{0,1\}^{4n}, \\ \boldsymbol{b}' &:= (\boldsymbol{b}_\ell, \boldsymbol{0}, \boldsymbol{b}_h, \boldsymbol{0}, \boldsymbol{b}_\ell, \boldsymbol{0}, \boldsymbol{b}_h, \boldsymbol{0}) \in \{0,1\}^{4n}, \end{aligned} \tag{5}$$

where $\boldsymbol{a} = \boldsymbol{a}_\ell \,||\, \boldsymbol{a}_h$ and $\boldsymbol{b} = \boldsymbol{b}_\ell \,||\, \boldsymbol{b}_h$, with subvectors of size $n/2$. Since each recursion splits each operand from n slots to $4n$ (as in (5)), the total number of required slots $S(n)$ satisfies the recurrence

$$S(n) = 4 \cdot S(n/2) \quad \text{for } n > 4, \quad \text{with } S(4) = 8.$$

Solving this recurrence, noting that $n = 2^k$, we have

$$S(n) = 4 \cdot 4 \cdots 4 = 4^{\log_2(n/4)} \cdot 8 = 4^{\log_2(n/4)} \cdot 8 = n^2/2.$$

Hence, an n-bit multiplication requires $n^2/2$ slots to be executed in parallel. □

This result is currently the main limitation of our work and implies that the concrete throughput of our approach is severely limited. We provide an high-level pseudo code of the proposed divide-and-conquer multiplication in Algorithm 6. We refer the interested reader to Appendix E.6 for a more detailed explanation of this algorithm.

Algorithm 6 Divide-and-conquer n-bit multiplication, high-level

Procedure $\mathsf{Mult}(\boldsymbol{x}, \boldsymbol{y}, n)$
  if $n = 4$ **then**
    return $\mathcal{I}$-to-$\mathcal{B}(\mathcal{B}$-to-$\mathcal{I}(\boldsymbol{x}) \cdot \mathcal{B}$-to-$\mathcal{I}(\boldsymbol{y}))$ ▷ *Multiplication in $\mathcal{I}$-CKKS*
  else
    $\boldsymbol{x}_\ell \leftarrow \boldsymbol{x}_{1,n/2}, \quad \boldsymbol{x}_h \leftarrow \boldsymbol{x}_{n/2,n} \quad \boldsymbol{y}_\ell \leftarrow \boldsymbol{y}_{1,n/2}, \quad \boldsymbol{y}_h \leftarrow \boldsymbol{y}_{n/2,n}$
    $\mathrm{term}_1 \leftarrow \mathsf{Add}(\mathsf{Mult}(\boldsymbol{x}_h, \boldsymbol{y}_h, n/2) \ll n, \mathsf{Mult}(\boldsymbol{x}_h, \boldsymbol{y}_\ell, n/2) \ll n/2)$
    $\mathrm{term}_2 \leftarrow \mathsf{Add}(\mathsf{Mult}(\boldsymbol{x}_\ell, \boldsymbol{y}_h, n/2) \ll n/2, \mathsf{Mult}(\boldsymbol{x}_\ell, \boldsymbol{y}_\ell, n/2))$
    return $\mathsf{BinBTS}(\mathsf{Add}(\mathrm{term}_1, \mathrm{term}_2))$

Lemma 3 (Multiplication cost). *An n-bit multiplication as in Algorithm 6 requires* $\log(n) - 1$ *bootstrappings under Table 4 parameters.*

The proof (in Appendix B.1) simply takes into account the number of recursive calls, i.e., $\log(n) - 2$, each taking one bootstrapping, plus the base case. Notice that we merge the four additions into one using the carry-save adder.

Logical shifts. Given that, to evaluate fast multiplications in parallel, we require each number to be followed by an amount of zeroes (larger than n), we can compute a logical shift as a rotation, thus (assuming LSB-first encoding):

$$\begin{aligned} x \cdot 2^s \equiv x \ll s \quad &\text{is implemented as } \mathrm{Rot}_{-s}(x) \\ x/2^s = x \cdot 2^{-s} \equiv x \gg s \quad &\text{is implemented as } \mathrm{Rot}_{s}(x). \end{aligned} \tag{6}$$

Some attention is required if, after this phase, we evaluate an addition. This might break the input hypothesis in Algorithm 3 that the addends are followed by a zero. In such case, we can simply perform a masking operation[6].

5 Non-linear function evaluation, in theory

As spoiled above, one of our goals is to be able to handle advanced arithmetic. In modern CPUs, to compute functions like square roots and quotients, a common pipeline is to first compute a rough estimation that is then refined by root finding algorithms. Currently, the only FHE library enabling (exact) division is TFHE-rs [Zam26, Section 3.17], and it is done that via a variant of the binary long division algorithm: this makes it by far the heaviest available operation in the library. On the other hand, we now suggest an approach which is closer to the workings of modern CPUs, enabled by our practical LUTs. Our high-level idea is to take advantage of LUTs to map the input to a first hint (or seed) for the root finding algorithm.

[6] In multiplications, this is not required as the extraction of high/low bits is already a masking itself

5.1 Warm-up

Assume we want to compute a non-linear function f that admits a closed-form, division-free Newton iteration with a simple update rule (e.g., $1/x$ or $1/\sqrt{x}$). An input x to the algorithm is an n-bit unsigned integer. In particular, we work with its normalization, denoted by $\hat{x}$. Any nonzero integer x can be normalized by shifting it left; if x has bit-length β, i.e., $x \in [2^{\beta-1}, 2^{\beta})$, define

$$\hat{x} = x \cdot 2^{n-\beta} \pmod{2^n}, \tag{7}$$

then, the MSB of $\hat{x}$ is at position $n-1$. Finally, notice that $\hat{x} \in [2^{n-1}, 2^n)$. We will later show (Section 6.1) how to compute such normalization concretely in discrete–CKKS. Our algorithm has two main stages:

- *Seed*: the $[2^{n-1}, 2^n)$ interval is split into 2^ℓ buckets of integers. A precomputed LUT stores, for each bucket, an integer close to the (scaled) target function on that bucket. The top ℓ bits of $\hat{x}$ select the corresponding bucket; the LUT hence gives a seed with a small error, ideally of $O(2^{-\ell})$.
- *Refinement*: an integer map Φ, built from Newton–Raphson's method and using only integer multiply/add/shift, is applied repeatedly: $x_{k+1} = \Phi(x_k)$. Each application roughly halves the relative error, i.e. doubles the number of correct bits, until the result is accurate to all n bits.

5.2 Constructing a LUT giving a good seed

Set a scaling exponent and define the *scaled target*:

$$F(\hat{x}) = \lfloor 2^{2n} f(\hat{x}) \rfloor, \qquad \text{with } \hat{x} \in [2^{n-1}, 2^n).$$

Let us first define ℓ as the bit size of the ℓ-to-n LUT; given our previous constraints (Section 3.3), we will assume from now on that $\ell = 8$. Let us now partition $[2^{n-1}, 2^n)$ into 2^ℓ buckets of integers, each of width $\delta = 2^{n-1-\ell}$. The bucket corresponding to $\hat{x}$ is indexed by $i = \lfloor (\hat{x} - 2^{n-1})/\delta \rfloor$, i.e. by reading off the top ℓ bits of $\hat{x}$ just below its leading 1. The LUT is thus defined as, given i, return $F(\sigma_i)$, with $\sigma_i = 2^{n-1} + i \cdot \delta$. We provide an illustration of the buckets splitting of $[2^{n-1}, 2^n)$ in Figure 6.

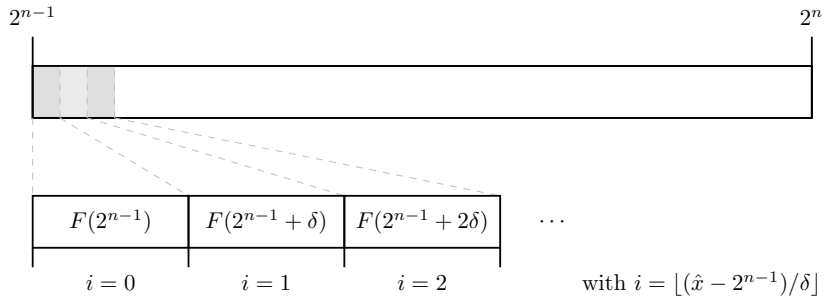


Fig. 6: Splitting the $[2^{n-1}, 2^n)$ interval in 2^ℓ buckets of size δ. Each i-th bucket covers $[\sigma_i, \sigma_i + \delta)$, returning $F(\sigma_i) = F(2^{n-1} + i \cdot \delta)$ on input i

To ease understanding, we report in Appendix F two numerical examples of the computation of seeds for the square root and quotient over 8 bits with a 4 bits LUT. We also provide in Table 3 a summary of the used variables, to help the reader. To begin with, we provide in Table 3 a summary of the used variables, to help the reader navigate the section. In order to study the seed

Table 3: Summary of the notation used throughout this section.

Symbol	Meaning
β	Bit length of x, i.e., the number of bits required to represent x
$\hat{x}$	Normalized input $\hat{x} = x \cdot 2^{n-\beta}$ that lies in $[2^{n-1}, 2^n)$
$F(x)$	The function to be evaluated, scaled by 2^{2n}, i.e, $F(x) = \lfloor 2^{2n} f(x) \rfloor$
ℓ	Bit size of the LUT (in this paper we assume $\ell = 8$)
δ	Width of each bucket between 2^{n-1} and 2^n (see Figure 6)
i	The index of the LUT slot corresponding to x, def. as $i = \lfloor (\hat{x} - 2^{n-1})/\delta \rfloor$
σ_i	The input of the LUT $\sigma_i = 2^{n-1} + i \cdot \delta$ corresponding to the index i
r	The root we want to compute (i.e., $r = 2^{2n}/\hat{y}$)
ε_i	The relative error at i-th Newton–Raphson iteration (8)

error throughout the iterations, let us first give it a name: we call it *seed relative error*:

$$\varepsilon_0 \;:=\; 1 - \frac{\lfloor 2^{2n} f(\sigma_i) \rfloor}{2^{2n} f(\hat{x})} = 1 - \frac{F(\sigma_i)}{2^{2n} f(\hat{x})}, \tag{8}$$

for $\hat{x}$ in bucket i. Since $\hat{x}$ is in bucket i, $0 \le \hat{x} - \sigma_i < \delta$, and since $\sigma_i \ge 2^{n-1}$, we can bound the ratio

$$0 \;\le\; \hat{x}/\sigma_i - 1 \;<\; 2^{-\ell}. \tag{9}$$

We now use (9) to bound ε_0 exactly for the division case. We defer the results for the square root in Appendix C for readability, but the execution is equivalent.

Corollary 4 (Division seed accuracy). *Let $f(x) = 1/x$. If $n > \ell$, then*

$$|\varepsilon_0| < 2^{-\ell}.$$

Proof. Remark the definition of ε_0 in (8). Write $\lfloor 2^{2n} f(\sigma_i) \rfloor = 2^{2n} f(\sigma_i) - \theta$ for some $\theta \in [0, 1)$. Observe that by definition of ε_0 we have that:

$$\varepsilon_0 = 1 - \frac{\lfloor 2^{2n} f(\sigma_i) \rfloor}{2^{2n} f(\hat{x})} = 1 - \frac{2^{2n} f(\sigma_i) - \theta}{2^{2n} f(\hat{x})} = 1 - \frac{\hat{x}}{\sigma_i} + \frac{\hat{x} \cdot \theta}{2^{2n}}$$

Notice that the second term, which we call t, is $0 \le t < 2^{-n}$. Also observe that by using (9), we obtain the following bound:

$$0 \le \hat{x}/\sigma_i - 1 < 2^{-\ell} \quad \xrightarrow{\text{Implies}} \quad -2^{-\ell} < 1 - \hat{x}/\sigma_i \le 0$$

thus giving

$$\varepsilon_0 = 1 - \hat{x}/\sigma_i + t > -2^{-\ell} + t \geq -2^{-\ell}$$

However, we also have that, since $1 - \hat{x}/\sigma_i \leq 0$, that $\varepsilon_0 \leq 0 + t < 2^{-n}$. Taking into account that $n > \ell$, we have that $2^{-n} < 2^{-\ell}$, so $\varepsilon_0 < 2^{-\ell}$. Combining with $\varepsilon_0 > -2^{-\ell}$ gives the result. □

5.3 Correctness of quotient

Given the initial seed $F(\sigma_i)$, along with its relative error ε_0, we now aim to refine the initial seed by iterating a map Φ built from Newton–Raphson's method applied to some $g(s) = 0$. We will from now on assume $f(x) = 1/x$, but the same reasoning can be done for square roots. Define

$$g(s) \;:=\; \frac{1}{s} - \frac{\hat{y}}{2^{2n}},$$

that has its root at $r := 2^{2n}/\hat{y}$ (i.e., the value we want to compute). Since its derivative $g'(s) = -1/s^2$, the update becomes

$$\Phi(s) = (s \cdot (2^{2n+1} - \hat{y} \cdot s))/2^{2n}. \tag{10}$$

In order to prove the correctness of our approach, we analyze the different $s_{j+1} := \lfloor \Phi(s_j) \rfloor$ produced this way, analyzing their distance to the root r through the j-th relative error

$$\varepsilon_j \;:=\; 1 - \frac{s_j}{r}, \quad \text{given the } j\text{-th iterate } s_j = r(1 - \varepsilon_j), \tag{11}$$

with ε_0 being the error present in the initial seed $s_0 = F(\sigma_i)$ of Corollary 4.

Lemma 4 (One Newton–Raphson step). *For every $j \geq 0$ there exists $\theta_j \in [0, 1)$ such that $s_{j+1} = r(1 - \varepsilon_j^2) - \theta_j$. In particular,*

$$0 \;\leq\; \varepsilon_{j+1} \;=\; \varepsilon_j^2 + \frac{\theta_j}{r} \;<\; \varepsilon_j^2 + \frac{\hat{y}}{2^{2n}}.$$

The proof follows by using $s_{j+1} = \lfloor \Phi(s_j) \rfloor$ and by replacing $s_j = r(1 - \varepsilon_j)$. It can be found in Appendix B.2. This lemma has two interesting insights: (i) the truncation to integers does not interfere with the quadratic convergence: the relative error is squared exactly, and the floor merely adds back at most one unit in the last place. (ii) $\varepsilon_{j+1} \geq 0$: from the first iteration onwards s_j is always an underestimate of $r = 2^{2n}/\hat{y}$. In practice, the quotient never overshoots $\lfloor x/y \rfloor$.

Lemma 5 (Accuracy after k steps). *Assume $\ell \geq 4$, $n \geq 8$ and $|\varepsilon_0| \leq 2^{-\ell}$. Then for every $j \geq 1$*

$$0 \;\leq\; \varepsilon_j \;\leq\; |\varepsilon_0|^{2^j} + 2 \cdot \frac{\hat{y}}{2^{2n}}.$$

What this lemma gives is that, starting from $|\varepsilon_0| < 2^{-\ell}$, we can reduce it exponentially fast, combining Corollary 4 and Lemma 4. In practice, the seed error is squared at every step, while the truncation error does not accumulate — it stays bounded by roughly one unit in the last place. The proof can be found in Appendix B.3.

Theorem 2 (Quotient correctness). *Let $\ell \geq 4$ and $n \geq 2\ell$ be powers of two. Given $x, y \in [1, 2^n)$, let $s_0 = F(\sigma_i)$ be the seed of Corollary 4. Let $s_1, \ldots, s_k$ with $k = \log(n/\ell)$ be the iterates produced by k applications of (10). Then,*

$$\left\lfloor \frac{x}{y} \right\rfloor - 1 \leq \left\lfloor x \cdot \frac{s_k}{2^{n+\beta}} \right\rfloor \leq \left\lfloor \frac{x}{y} \right\rfloor.$$

The proof (Appendix B.4) is derived by combining all the previous building blocks, and shows that both error sources — the residual Newton error, below 2^{-n} by quadratic convergence, and the accumulated floors — contribute at most $2^{1-\beta}$ once scaled by the quotient, so the estimate misses $\lfloor x/y \rfloor$ by less than one. Recovering $\lfloor x/y \rfloor$ exactly therefore costs a single conditional increment.

6 Non-linear function evaluation, in practice

6.1 Bit length

Previously, we assumed $\hat{x} \in [2^{n-1}, 2^n)$ to be available, given $x \in [0, 2^n)$. This is, however, a strong assumption since computing $\hat{x} = x \cdot 2^{n-\beta}$ requires to count the bit length of x (namely β). To the best of our knowledge, this operation is only implemented in TFHE (called *leading ones*, or *bit length* in Python). We provide way to compute it in logarithmic complexity, using only CKKS primitives in Algorithm 7. Notice that the procedure performs $\log(n)$ OR evaluations, which

Algorithm 7 Bit length (or leading one position)

Input: A $\mathcal{B}$-CKKS ciphertext $\boldsymbol{x}$ encoding an n-bit number, followed by n zeros
Output: An $\mathcal{I}$-CKKS ciphertext containing the bit length β of $\boldsymbol{x}$ in the first ℓ slots

1: **Procedure** $\mathsf{BitLength}(\boldsymbol{x}, n, \ell)$
2: **for** $i \leftarrow 0$ **to** $\log(n) - 1$
3: $\boldsymbol{x} \leftarrow \boldsymbol{x} \vee \mathsf{Rot}_{2^i}(\boldsymbol{x})$ ▷ *Filling with ones up to the leading one*
4: $\boldsymbol{x} \leftarrow \mathsf{RotateAndSum}(\boldsymbol{x})$ ▷ *Count how many ones there are into x_0*
5: $\boldsymbol{x} \leftarrow$ repeat x_0 in the first ℓ slots
6: **return** $\boldsymbol{x}$

in turn means that its complexity is of $\log(n)$ multiplications ($x \vee y$ is computed as $(x + y) - (x \cdot y)$). Also, observe that the result of Algorithm 7 (after repeating the integer value in ℓ slots) is an $\mathcal{I}$-CKKS ciphertext which can be decomposed à la Algorithm 1. This allows to compute $\mathsf{Bits}(\beta)$, as required in the following step. Indeed, to compute the actual $\hat{x}$, we are required to shift x by $n - \beta$, which is encrypted because of β.

6.2 Blind rotations

This can be done via a *blind rotation*, namely shifting the bits of a ciphertext by an encrypted value. This is well-known in the DM/CGGI community, although their study under CKKS, to the best of our knowledge, started only recently in [CHKS25]. It turns out that our radix-2 representation well fits such rotations. The main idea is as follows: since the rotation index r is binary decomposed: iterate over each bit and transform the input $\boldsymbol{x}$ as follows $\boldsymbol{x} \leftarrow \boldsymbol{x} \cdot (1 - r_i) + \mathrm{Rot}_{2^i}(\boldsymbol{x}) \cdot r_i$, where r_i is the i-th bit of r. In practice, r_i behaves as a selection variable, and since it encodes the value of $\boldsymbol{r}$, it enables to compute $\mathrm{Rot}_r(\boldsymbol{x})$. We report in Algorithm 8 the procedure used to perform such rotation. Of course,

Algorithm 8 Blind rotation

Input: Two $\mathcal{B}$-CKKS ciphertexts: $\boldsymbol{x}$ encoding an n-bit number and $\boldsymbol{r}$ encoding a k-bit rotation index, with $k \leq n$
Output: A $\mathcal{B}$-CKKS ciphertext containing $x \gg r$

Procedure $\mathsf{BlindRotation}(\boldsymbol{x}, \boldsymbol{r}, n, k)$
  for $i \leftarrow 0$ **to** $k - 1$
    $\hat{\boldsymbol{r}} \leftarrow r_i$ repeated along all the n bits
    ▷ *If* $r_i = 1$ *then rotate by* 2^i *positions, else do nothing.*
    $\boldsymbol{x} \leftarrow \boldsymbol{x} \cdot (1 - \hat{\boldsymbol{r}}) + \mathsf{Rot}_{2^i}(\boldsymbol{x}) \cdot \hat{\boldsymbol{r}}$
return $\boldsymbol{x}$

by rotating using the inverse index -2^i, we can obtain the opposite shift $x \ll r$.

6.3 Computing the quotient, in practice

We now finally have all the ingredients to compute the (exact) n-bit division operation in CKKS, namely $q = \lfloor x/y \rfloor$ such that $qy + r = x$ for some r. The main algorithm is composed of three steps: (i) the seed is computed via a ℓ-to-$(n+2)$ LUT, (ii) the obtained result is refined via Newton–Raphson iterations and lastly (iii) the correct result is recovered by reverting the normalization performed in the first step. We present the whole procedure in Algorithm 9. Of course, observe that if the denominator is a plaintext (known constant), then the division complexity is roughly the same as a multiplication (the scaled $2^{2n}/b$ is precomputed in the clear) as only *Stage 3/4* must be performed under CKKS. Notice that we need $y > 0$ to connect to Theorem 2, although on input $y = 0$ the algorithm will simply return 0.

The square root case. Square roots are computed similarly as shown in Algorithm 9, and Corollary 5 ensures that the LUT error is $O(2^{-\ell-1})$. However, for the final step (in Algorithm 9 that corresponds to line 13), observe that $x/(y \cdot 2^{n+\beta})$, becomes $x/y \cdot 2^{-(n+\beta)}$. For the square root case, the same leads to $\sqrt{x \cdot 2^{n+\beta}}$, which becomes $\sqrt{x} \cdot 2^{\lfloor n+\beta \rfloor/2}$, so we must take into account the two

Algorithm 9 Integer quotient $\lfloor x/y \rfloor$

Input: Word size n; LUT of size 2^ℓ; dividend x, divisor y with $0 < x, y < 2^n$; precomputed LUT $F(0), \ldots, F(2^\ell - 1)$ with $F(i) = \left\lfloor 2^{2n}/(2^{n-1} + i \cdot 2^{n-1-\ell}) \right\rfloor$
Output: $\lfloor x/y \rfloor$

Procedure Quotient($\boldsymbol{x}, \boldsymbol{y}, n$)
  $\beta \leftarrow$ bit length of y ▷ *Algorithm 7*
  $\hat{y} \leftarrow y \ll n - \beta$ ▷ *Algorithm 8*
Stage 1: ℓ-to-$(n+2)$ LUT (seed)
  $i \leftarrow \hat{y} \gg (n - 1 - \ell) \bmod 2^\ell$ ▷ *Bucket index*
  $s \leftarrow F(i)$ ▷ *Algorithm 2*
Stage 2: Newton–Raphson refinement
  $k \leftarrow \log_2(n/\ell)$ ▷ *Iterations needed for full n-bit accuracy*
  for $j \leftarrow 1$ **to** k
    ▷ *In a practical implementation, we split the computation to work over n bits*
    $s \leftarrow \left(s \cdot (2^{2n+1} - \hat{y} \cdot s)\right) \gg 2n$
Stage 3: recover the quotient
  $q \leftarrow (x \cdot s) \gg n + \beta$ ▷ *Undo the normalization shift, variant of Algorithm 8*
Stage 4: single conditional correction
  $\rho \leftarrow x - q \cdot y$ ▷ $\rho \in [0, 2y)$
  return $q + (\rho \geq y)$ ▷ *Last conditional increment, see Theorem 2*

cases where $n + \beta$ is odd or even. This requires a minor change and does not change the overall complexity of the procedure. The workflow is similar to the one for the division (i.e., the practical complexity is comparable), and requires one less Newton–Raphson iteration. However the required lemmas (along with their proofs) are slightly more involved. It is fully described in Appendix C.

7 Experiments

We implemented our framework using (a custom fork of) OpenFHE [AB+22, KPP22] and the code is publicly available[7]. The repository provides an equivalent implementation using Python notebooks with plaintext vectors, intended for didactic purposes. A GPU version, based on [ADVLG+25], is publicly available too[8]. The CPU experiments have been executed on a Macbook M5 Max with 36 GB of memory using the parameters in Table 4 supporting 128-bit of security according to the $\log(QP) < 1553$ threshold in [BMTH21, Table 3]. Observe that the available depth leaves 15 levels of arbitrary computations outside of the bootstrapping, enabling the practical evaluation of $\mathcal{R}$-CKKS circuits prior or after the computations in $\mathcal{B}$-CKKS. Of course [GZ26] could use a deeper moduli chain, at the cost of increasing the overall latency. We also provide, in Table 5, the number of required call to bootstrapping for each integer operation. Notice that

[7] github.com/lorenzorovida/advanced-arithmetic-fhe
[8] github.com/lorenzorovida/advanced-arithmetic-fhe-cuda

Table 4: The CKKS parameters used for the experiments. $h, \hat{h}$ are the Hamming weights of the dense/sparse encapsulated key [BTPH22], respectively

							Multiplicative depth in $\mathbb{R}$		
	$\log(N)$	$\log(\Delta)$	$\log(q_i)$	$\log(QP)$	d_{num}	$(h, \hat{h})$	Total	BTS	Arbitrary
Ours	16	41	36	1542	3	(192, 32)	29	14	15
[GZ26]	16	43	43	1254	3	(192, 32)	19	13	6

Table 5: Some integer operations, and their complexity in terms of the number of bootstrappings, compared to [GZ26].

		Add	Mult	Shift	Equal	Boolean	Division	Square root
32-bit	Ours	1	4	0	1	Leveled	30	43
	[GZ26]	Leveled	2	8	9	8	*Not available*	*Not available*
64-bit	Ours	1	5	0	1	Leveled	48	68
	[GZ26]	Leveled	2	16	17	16	*Not available*	*Not available*
128-bit	Ours	1	6	0	1	Leveled	70	99
	[GZ26]	Leveled	2	32	33	32	*Not available*	*Not available*

on logical operations our bootstrapping count is much smaller than [GZ26], while on arithmetic operations is slightly larger. This confirms our initial hypotheses.

Next, we report in Figure 7, some runtimes over CPU on some of the various supported operations in our framework. Subsequently, we report in Table 6 the obtained results in CPU compared to the state of the art discrete–CKKS proposal [GZ26]. We additionally report, later in the appendix (Table 10) some heuristic results about the noise analysis of all the proposed operations.

Comparison with [GZ26]. Thanks to their convenient public code, we evaluated [GZ26] on our machine. Results confirm that [GZ26] is slightly faster on arithmetic operation but dramatically slower when extending the set of operations to non-arithmetic ones. Moreover, the moduli chain in [GZ26] is short and makes computations in $\mathcal{R}$-CKKS prior or after impractical. Fixing this issue would require a larger modulus, implying slower overall performance. The results confirm that our framework better fits use cases where arithmetic and non arithmetic operations are mixed, and not massive amount of data is processed, while [GZ26] fits cases where mostly arithmetic operations are evaluated. We report in Appendix A.5 further comparisons about the throughput.

8 Conclusion

We presented a framework that enables efficient Boolean, integer and real computations in CKKS. Compared to similar works, it is comparable in terms of latency of arithmetic operations but much faster on non arithmetic operations.

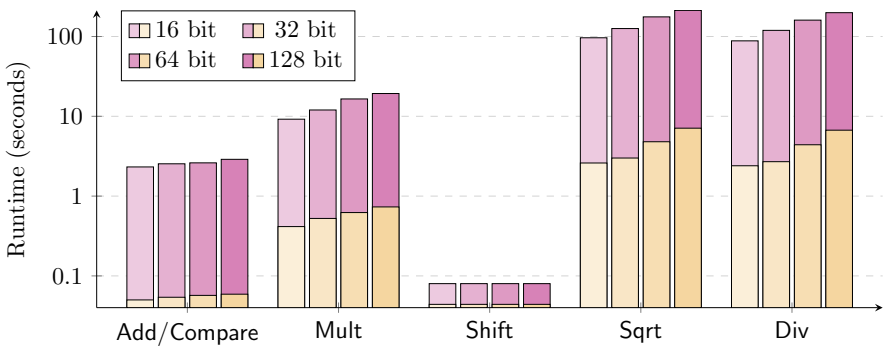


Fig. 7: Runtime data for some of the integer operations supported in our scheme for different word sizes, pink for CPU, amber for GPU. The y axis is in log-scale

Plus, we enable for the first time the native computation of exact quotient and square root over large integers within discrete–CKKS. We see this as a step towards closing the gap between the functionalities offered by TFHE and CKKS.

Limitations and future work. Our throughput is severely limited by Corollary 3. By relaxing the constraint posed by the multiplication algorithm, it would be possible to dramatically improve the packing ability of our scheme. Theoretically, the optimal capacity of the scheme — considering binary decomposed values — would be of s/n slots, with s being the number of CKKS available slots and n the number of bits. Hence, our current $2s/n^2$ packing limit is suboptimal by a factor $n/2$. Exploring different way to evaluate the $\lfloor x/2^i \rfloor \bmod 2$ decomposition seems like a natural follow-up. One of our limitations is indeed that the approximation region is $[0, 256)$, but evaluating the trigonometric polynomial in $e^{2\pi i x}$ basis (as, e.g., in [AKP25]) might unlock practical decomposition in intervals larger than $[0, 256)$.

Acknowledgements. This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union — NextGenerationEU. L. Rovida would like to thank A. Alexandru (Duality Tech.) for her help concerning the implementation of cleaning bootstrapping in OpenFHE, E. Sokli (University of Milan-Bicocca) for giving insights on DFT and J. Kim (Stanford University) for answering some questions about [Kim25a, Kim25b]. Claude free has been used to port the CPU code into CUDA, the author solely checked the validity of the results.

References

AB+22. Ahmad Al Badawi et al. OpenFHE: Open-Source Fully Homomorphic Encryption Library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 53–63, 2022.

Table 6: Comparison of latency in different integer operations in CPU. Non arithmetic operations in [GZ26] require the A2B+B2A subroutines

	32 bits		64 bits		128 bits	
	Ours	[GZ26]	Ours	[GZ26]	Ours	[GZ26]
Arithmetic						
$a + b$	2.38 s	**Leveled**	2.42 s	**Leveled**	2.98 s	**Leveled**
$a \cdot b$	11.5 s	**9.0 s** (0.8×)	15.6 s	**12.4 s** (0.8×)	18.8 s	**14.5 s** (0.7×)
Non arithmetic						
$a > b$	**2.40 s**	26.0 s (10.8×)	**2.45 s**	51.5 s (21.1×)	**3.01 s**	101 s (33.5×)
$a = b$	**2.48 s**	31.5 s (12.7×)	**2.80 s**	56.0 s (20.0×)	**2.89 s**	105 s (36.3×)
$a \ll b$	**0.08 s**	26.5 s (331×)	**0.08 s**	52 s (650×)	**0.08 s**	103 s (1287×)
Advanced arithmetic						
a/b	**94.4 s**	*Not available*	**153 s**	*Not available*	**225 s**	*Not available*
$\sqrt{a}$	**127 s**		**206 s**		**298 s**	
Slots	64	2048	16	1024	4	512

ABC+25. Daniel Aronoff, Adithya Bhat, Panagiotis Chatzigiannis, Mohsen Minaei, Srinivasan Raghuraman, Robert M. Townsend, and Nicolas Xuan-Yi Zhang. SoK: Fully-homomorphic encryption in smart contracts. Cryptology ePrint Archive, Paper 2025/527, 2025.

ADVLG+25. Carlos Agull-Domingo, scar Vera-Lpez, Seyda Guzelhan, Lohit Daksha, Aymane El Jerari, Kaustubh Shivdikar, Rashmi Agrawal, David Kaeli, Ajay Joshi, and Jos L. Abelln. FIDESlib: A Fully-Fledged Open-Source FHE Library for Efficient CKKS on GPUs. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Ghent, Belgium, 2025. IEEE. Poster paper.

AKP25. Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using ckks. In *Advances in Cryptology – CRYPTO 2025*, pages 304–337, 2025.

AZR20. Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. Uniswap Whitepaper, March 2020.

BCKS24. Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with ckks. In *Advances in Cryptology – EUROCRYPT 2024*, pages 94–123, 2024.

BCL+25. Loris Bergerat, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. TFHE gets real: an efficient and flexible homomorphic floating-point arithmetic. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):126–162, 2025.

BCM+25. Jean-Philippe Bossuat, Anamaria Costache, Christian Mouchet, Lea Nrnberger, and Juan Ramn Troncoso-Pastoriza. Accurate and composable noise estimates for CKKS with application to exact HE computation. 2(2), 2025.

BFG+26. Zvika Brakerski, Offir Friedman, Daniel Golan, Alon Gurny, Dolev Mutzari, and Ohad Sheinfeld. REFHE: Fully Homomorphic ALU. In *Advances in Cryptology EUROCRYPT 2026*, pages 183–213, 2026.

BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 309–325, 2012.

BK25. Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In *Advances in Cryptology – CRYPTO 2025*, pages 338–370, 2025.

BKSS24. Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In *Advances in Cryptology – ASIACRYPT 2024*, pages 330–360, 2024.

BMTH21. Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Advances in Cryptology – EUROCRYPT 2021*, pages 587–617, 2021.

Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Advances in Cryptology – CRYPTO 2012*, pages 868–886, 2012.

BTPH22. Jean-Philippe Bossuat, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In *Applied Cryptography and Network Security*, pages 521–541, 2022.

CCKS23. Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for ckks and reduced modulus consumption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 696–710, 2023.

CCP+24. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the ind-cpad security of exact fhe schemes. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, page 25052519, 2024.

CCS19. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2019*, pages 34–54, 2019.

CCSH+25. Nicholas Carlini, Jorge Chvez-Saab, Anna Hambitzer, Francisco Rodrguez-Henrquez, and Adi Shamir. Polynomial Time Cryptanalytic Extraction of Deep Neural Networks in the Hard-Label Setting. In *Advances in Cryptology EUROCRYPT 2025*, pages 364–396, 2025.

CGGI16. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *Advances in Cryptology – ASIACRYPT 2016*, pages 3–33, 2016.

CHK+18. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2018*, pages 360–384, 2018.

CHKS25. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A Shallow and Highly Parallelizable CKKS Bootstrapping Algorithm. In *Advances in Cryptology EUROCRYPT 2025*, pages 398–428, 2025.

CJKR26a. Jung Hee Cheon, Daehyun Jang, Jaehee Kang, and Hanee Rhee. Private embedding lookup with encrypted compact queries under fully homomorphic encryption, 2026.

CJKR26b. Jung Hee Cheon, Daehyun Jang, Jaehee Kang, and Hanee Rhee. SWIFT: Shallow and SIMD-aware CKKS functional bootstrapping for low-latency. Cryptology ePrint Archive, Paper 2026/1163, 2026.

CJM20. Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic Extraction of Neural Network Models. In *Advances in Cryptology CRYPTO 2020*, pages 189–218. Springer International Publishing, 2020.

CJP21. Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning*, pages 1–19, 2021.

CKKL24. Heewon Chung, Hyojun Kim, Young-Sik Kim, and Yongwoo Lee. Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. Cryptology ePrint Archive, Paper 2024/274, 2024.

CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437, 2017.

CLPX18. Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In *Cryptographers Track at the RSA Conference*, pages 116–136, 2018.

CPL26. Gyeongwon Cha, Dongjin Park, and Joon-Woo Lee. Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. In *Advances in Cryptology – EUROCRYPT 2026*, pages 243–273, 2026.

DAP+26. Jules Dumezy, Andreea Alexandru, Yuriy Polyakov, Pierre-Emmanuel Clet, Olive Chakraborty, and Aymen Boudguiga. Evaluating larger lookup tables using ckks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(1):559591, 01 2026.

DM15. Léo Ducas and Daniele Micciancio. Fhew: Bootstrapping homomorphic encryption in less than a second. In *Advances in Cryptology – EUROCRYPT 2015*, pages 617–640, 2015.

DMPS23. Nir Drucker, Guy Moshkowich, Tomer Pelleg, and Hayim Shaul. Bleach: Cleaning errors in discrete computations over ckks. *J. Cryptol.*, 37(1), 11 2023.

DS26. Jules Dumezy and Elias Suvanto. Character block encodings for discrete CKKS: Single-level LUTs and low-depth arithmetic. Cryptology ePrint Archive, Paper 2026/1200, 2026.

FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012.

Gen09. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC '09*, pages 169–178, 2009.

GV25. Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In *Advances in Cryptology EUROCRYPT 2025*, pages 366–397, 2025.

GZ26. Mingyu Gao and Hongren Zheng. FHE for SIMD Arithmetic Logic Units with Amortized O(1) Bootstrapping per Ciphertext. In *Advances in Cryptology – CRYPTO 2026*, pages 299–330, 2026.

IMT25. Akira Ito, Takayuki Miura, and Yosuke Todo. Is the hard-label cryptanalytic model extraction really polynomial? Cryptology ePrint Archive, Paper 2025/1868, 2025.

Int. Intel. Intel 64 and IA-32 Architectures Software Developer's Manual. Technical report, Intel Corporation. Available online: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

JPK+24. Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4361–4375, 2024.

Kim25a. Jaehyung Kim. Efficient homomorphic integer computer from ckks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):873898, 9 2025.

Kim25b. Jaehyung Kim. Faster homomorphic integer computer. Cryptology ePrint Archive, Paper 2025/1440, 2025.

KN25. Jaehyung Kim and Taeyeong Noh. Modular reduction in CKKS. *IACR Communications in Cryptology*, 2(2), 2025.

KPP22. Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. Approximate homomorphic encryption with reduced approximation error. In *Topics in Cryptology CT-RSA 2022: Cryptographers Track at the RSA Conference 2022*, pages 120–144, 2022.

KS73. Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Trans. on Comp.*, C-22(8):786–793, 1973.

KYML25. Seunghu Kim, Eymen nay, Ayse Yilmazer-Metin, and Hyung Tae Lee. Optimized rank sort for encrypted real numbers. Cryptology ePrint Archive, Paper 2025/1170, 2025.

LM21. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In *Advances in Cryptology – EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, page 648677, 2021.

LPR13a. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *J. ACM*, 60(6), 11 2013.

LPR13b. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-lwe cryptography. In *Advances in Cryptology – EUROCRYPT 2013*, pages 35–54, 2013.

LSE+26. Haolin Liu, Adrien Siproudhis, Samuel Experton, Peter Lorenz, Christina Boura, and Thomas Peyrin. Navigating the deep: End-to-end extraction on deep neural networks. In *Advances in Cryptology – EUROCRYPT 2026*, pages 482–512, 2026.

MEHP25. Federico Mazzone, Maarten Everts, Florian Hahn, and Andreas Peter. Efficient ranking, order statistics, and sorting under ckks. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 8541–8558, 2025.

NHY+25. Chao Niu, Zhicong Huang, Zhaomin Yang, Yi Chen, Liang Kong, Cheng Hong, and Tao Wei. XBOOT: Free-XOR gates for CKKS with applications to transciphering. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):118–144, 09 2025.

Pas26. Jonathan Passerat-Palmbach. Leveraging homomorphic encryption for maximal extractable value (mev) mitigation: Blind arbitrage on decentralised exchanges. In *Financial Cryptography and Data Security*, pages 402–418, 2026.

PS73. Michael S Paterson and Larry J Stockmeyer. On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM Journal on Computing*, 2(1):60–66, 1973.

PZZ26. Chris Peikert, Doron Zarchy, and Guy Zyskind. High-Precision Exact FHE Made Simple, General, and Fast. In *Advances in Cryptology – CRYPTO 2026*, pages 525–557, 2026.

Reg09. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), 9 2009.

RLB26. Lorenzo Rovida, Alberto Leporati, and Simone Basile. Lightweight sorting in approximate homomorphic encryption. *IACR Communications in Cryptology*, 3(1), 2026.

SB02. Josef Stoer and R. Bulirsch. *Introduction to Numerical Analysis*, volume 12. Springer, 3 edition, 2002.

SV65. A. Sharma and A. K. Varma. Trigonometric interpolation. *Duke Mathematical Journal*, 32(2):341 – 357, 1965.

Tre19. Lloyd N. Trefethen. *Approximation Theory and Approximation Practice, Extended Edition*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2019.

Zam22. Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. https://github.com/zama-ai/tfhe-rs.

Zam26. Zama. TFHE-rs: A (practical) handbook — first edition. Technical report, Zama, February 2026. Available at https://github.com/zama-ai/tfhe-rs-handbook/blob/main/tfhe-rs-handbook.pdf.

ZEL+25. Guy Zyskind, Yonatan Erez, Tom Langer, Itzik Grossman, and Lior Bondarevsky. Fhe-rollups: Scaling confidential smart contracts on ethereum and beyond. In *Proceedings of the 6th ACM International Symposium on Blockchain and Secure Critical Infrastructure*, page 19, 2025.

ZYZ+24. Fahong Zhang, Chen Yang, Rui Zong, Xinran Zheng, Jianfei Wang, and Yishuo Meng. An efficient and scalable fhe-based pdq scheme: Utilizing fft to design a low multiplication depth large-integer comparison algorithm. *IEEE Transactions on Information Forensics and Security*, 19:2258–2272, 2024.

A Extended comparison with other large integers schemes

We provide in Table 7 a *very high-level* overview of some of the current schemes, and how they compare in terms of available operations and latency/SIMD performance. Since that table compresses a large amount of information, we now provide a more detailed justification by presenting a brief introduction to each of the referenced schemes (excluding the ones already discussed in Section 1.2).

Table 7: A very high level comparison with some new FHE schemes over integers and their available operations. (†)Latency is the most competitive on small plaintext spaces, but it increases quadratically when enlarging the plaintext space. ✓ Supported, ≈ supported but not efficient, · not natively supported. Performance is a qualitative assessment from "poor" (○○○) up to "excellent" (●●●)

	Binary			Integer						Real		Performance	
	$\wedge$	$\vee$	$\oplus$	$+$	$\times$	$\ll$	$>$	$\div$	$\sqrt{}$	$+$	$\times$	Latency	SIMD
DM/CGGI													
[Zam22]	✓	✓	✓	✓	✓	✓	✓	✓	·	·	·	●●○†	·
BGV/BFV–like													
[GV25]	·	·	·	✓	✓	·	·	·	·	·	·	●●◐	●◐○
[BFG+26]	✓	✓	✓	✓	✓	✓	·	·	·	·	·	●●○	·
[PZZ26]	≈	≈	≈	✓	✓	✓	≈	·	·	·	·	●●◐	●◐○
Discrete–CKKS													
[BK25]	·	·	·	✓	✓	·	·	·	·	✓	✓	◐○○	●◐○
[Kim25b]	·	·	·	✓	✓	≈	≈	·	·	✓	✓	●○○	●●◐
[CPL26]	·	·	·	✓	✓	≈	✓	·	·	✓	✓	●◐○	●●○
[GZ26]	≈	≈	≈	✓	✓	≈	≈	·	·	✓	✓	●●○	●●○
Ours	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	●●○	◐○○

A.1 DM/CGGI

Binary gates are TFHE core primitive via programmable bootstrapping. Integer addition, multiplication, shifts and comparison are enabled by simulating binary operations. Division is supported in TFHE-rs [Zam26, Section 3.17]. As for square root, there is a bounty program[9] that lists $\sqrt{x}$ as a feature still to be built, which directly confirms it isn't in the library today to the best of our knowledge. We note that there is work in progress to implement floating point arithmetic in TFHE, refer to, e.g., [BCL+25].

A.2 BFV-like

Generalized BFV (GBFV). [GV25] is explicitly framed as combining "the SIMD capabilities of BFV with the low noise growth of CLPX" while remaining "efficiently bootstrappable". This is a native BFV-family scheme: addition and multiplication are supported as the baseline plaintext-ring operations. More advanced operations require further constructions which are not discussed in [GV25].

[9] https://github.com/zama-ai/bounty-program/issues/142

REFHE. [BFG+26] builds on the BGV framework and arithmetic operations (modulo 2^n) are supported natively. The bootstrapping algorithm has the feature that along the way it decomposes machine words into bits, so that during bootstrapping it is possible to perform logical operations (functional bootstrapping). This confirms availability for [BFG+26] for binary and logical operations. However, the linear cost for logic operations makes them not very efficient, compared to e.g., TFHE.

Decomposed BFV (DBFV). [PZZ26] targets arithmetic for a huge prime or power-of-two modulus p, and is purely about closing the gap between fresh-ciphertext error rates (or multiplication noise growth) and p for exact schemes. Concretely, a plaintext of $\mathbb{Z}_p$ is represented by a relaxed base-b decomposition, and a ciphertext is essentially $d \approx \log_b p$ ordinary BFV ciphertexts encrypting the digits. Homomorphic addition and multiplication are then plain bivariate polynomial operations on lifted ciphertexts, so both the fresh error rate and the multiplicative error growth behave like BFV with plaintext modulus $d \cdot p^{1/d}$ rather than p. In particular, additions and multiplications are very cheap, although the bootstrapping seems not as efficient as the TFHE or CKKS one.

A.3 Discrete–CKKS

Gao–Zheng (GZ). [GZ26] introduces a new scheme designed to efficiently support both arithmetic and Boolean operations on SIMD-packed machine words. Existing approaches force a tradeoff: scalar schemes like DM/CGGI and REFHE handle bit operations natively but pay quadratic or linear bootstrapping cost in the word size, while SIMD digit-based schemes like [CPL26] achieve logarithmic cost but still require iterative bootstrapping to convert between arithmetic and boolean representations. The beautiful key innovation in [GZ26] is the *triangle encoding*: a middle ground between arithmetic representations and decomposed bit vectors. For the arithmetic-to-boolean conversion, they observe that each iteration only touches a thin "slice" of each machine word, so multiple ciphertexts can be merged and processed together in one bootstrap, amortizing the cost to a constant number of bootstraps per ciphertext when enough data is available (i.e., their batched B-A2B algorithm).

Differently from [GZ26] — whose goal is to target massive amount of data — our work better fits use cases where the number of input values is not huge (e.g., 64 slots of 32-bits numbers) and, moreover, the circuit is composed of a mixture of arithmetic and Boolean operations, as we do not require conversions between the two representations. A conceptual comparison is given in Figure 8. To summarize, although [GZ26] is characterized by an encoding that allows to perform fast high-precision arithmetic, its main bottleneck is the transformation to Boolean. Our proposal avoids this transformation completely by storing values directly as Boolean values. This comes at the cost of having less available slots and slightly slower arithmetic operations.

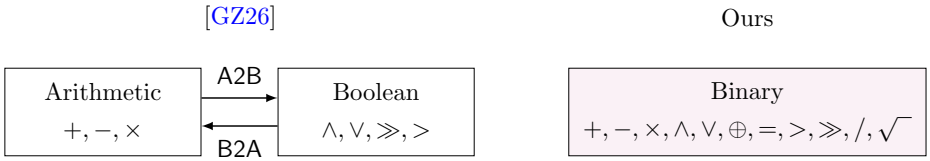


Fig. 8: Conceptual comparison between [GZ26] and our work. We do not need to switch between *arithmetic* and *Boolean* as our operations are natively based on binary values (i.e., they work in $\mathcal{B}$-CKKS). Figure inspired by [GZ26] slides used at CRYPTO '26

A.4 Extended GPU comparison with TFHE-rs

We lastly report additional GPU results in Table 8. (same environment as in Table 1). Note that Ours processes 64 slots for 32-bit, 16 slots for 64-bit, and 4 slots for 128-bit integers, while TFHE-rs always one.

A.5 Extended throughput comparison with [GZ26]

To complete the comparison in Section 7, we provide in Table 9 some numbers that confirm that [GZ26] typically behaves better in terms of throughput, namely when massive batches of data are processed at the same time. Nonetheless, we conjecture that our method can still perform faster if primitives as square root or quotient are involved in the homomorphic operations.

B Deferred proofs

We now present the proofs avoided in the main body to improve readability.

B.1 Lemma 3 (Multiplication cost)

Proof. Each recursive call evaluates the CSA (line 18 of Algorithm 12), which requires $6+\log(n)$ multiplicative levels (see (29)). Combining this with the masking phase (consuming 1 level $\log(n)$ times) we have that the recursive calls use

$$\log(n)(1+6+\log(n)) = 7\log(n) + (\log(n))^2$$

multiplicative levels. Lastly, the base case depends the approximation of the bit decomposition functions, but we assume to be able to evaluate it within 15 levels (Table 4). We report that in our proof of concept, a degree 451 Chebyshev polynomial (requiring 9 multiplicative levels) has maximum error $\ell_\infty < 10^{-7}$. In general, the available multiplicative depth in Table 4 is 15, and up to $n = 256$ we can evaluate each recursive call (using $7 + \log(n)$ levels) followed by a bootstrapping. Since the recursive calls are $\log(n) - 2$, and the base case requires one bootstrapping after the evaluation of the bit-extraction function, the $\log(n) - 1$ result follows. □

Table 8: Operations per second and latency per slot of our proof of concept and TFHE-rs [Zam22]. *Evaluation followed by a bootstrapping.

Operations per second				
Operation	Method	32-bit	64-bit	128-bit
Addition (+)	Ours	**1185 ops/s**	**281 ops/s**	**65.6 ops/s**
	[Zam22]	8.5 ops/s (138×)	6.8 ops/s (41×)	4.9 ops/s (13×)
Multiplication (×)	Ours	**121.9 ops/s**	**25.7 ops/s**	**5.46 ops/s**
	[Zam22]	2.7 ops/s (45×)	1.17 ops/s (22×)	0.48 ops/s (11×)
Boolean ($\wedge, \vee, \oplus$)	Ours*	**4000 ops/s**	**1000 ops/s**	**250 ops/s**
	[Zam22]	34.5 ops/s (116×)	34.5 ops/s (29×)	34.5 ops/s (7×)
Quotient (/)	Ours	**23.7 ops/s**	**3.64 ops/s**	**0.597 ops/s**
	[Zam22]	0.12 ops/s (201×)	0.07 ops/s (51×)	0.023 ops/s (22×)

Latency/Slots				
Operation	Method	32-bit	64-bit	128-bit
Addition (+)	Ours	**0.84 ms**	**3.5 ms**	**15 ms**
	[Zam22]	117 ms (138.7×)	146 ms (41.0×)	204 ms (13.4×)
Multiplication (×)	Ours	**8.2 ms**	**39 ms**	**183 ms**
	[Zam22]	370ms (45.1×)	852 ms (21.9×)	2073 ms (11.3×)
Boolean ($\wedge, \vee, \oplus$)	Ours*	**0.25 ms**	**1 ms**	**4 ms**
	[Zam22]	29 ms (116.0×)	29 ms (29.0×)	29 ms (7.25×)
Quotient (/)	Ours	**42.19 ms**	**275 ms**	**1675 ms**
	[Zam22]	8500 ms (201.5×)	14000 ms (50.9×)	36800 ms (22.0×)

B.2 Lemma 4 (One Newton–Raphson step)

Proof. Remark that, by definition $\Phi(s_j) = s_j(2^{2n+1} - \hat{y}\, s_j)/2^{2n}$. We can substitute $s_j = r(1-\varepsilon_j)$ and $\hat{y} = 2^{2n}/r$ to obtain:

$$\Phi(s_j) = \frac{r(1-\varepsilon_j)(2^{2n+1} - \frac{2^{2n}}{r} \cdot r(1-\varepsilon_j))}{2^{2n}} = r(1-\varepsilon_j)\big(2-(1-\varepsilon_j)\big) = r(1-\varepsilon_j^2).$$

However, we have that $s_{j+1} = \lfloor \Phi(s_j) \rfloor$. Therefore, we define $s_{j+1} = r(1-\varepsilon_j^2) - \theta_j$ for some $\theta_j \in [0, 1)$.Then

$$\varepsilon_{j+1} = 1 - \frac{s_{j+1}}{r} = \varepsilon_j^2 + \frac{\theta_j}{r} \geq 0.$$

Table 9: Comparison of latency/slots in different integer operations in CPU. Non arithmetic operations in [GZ26] require the A2B+B2A subroutines

	32 bits		64 bits		128 bits	
	Ours	[GZ26]	Ours	[GZ26]	Ours	[GZ26]
Arithmetic						
$a+b$	37 ms	**Leveled**	151 ms	**Leveled**	745 ms	**Leveled**
$a \cdot b$	180 ms	**4 ms** (.02×)	975 ms	**12 ms** (.01×)	4.8 s	**28 ms** (.006×)
Non arithmetic						
$a > b$	37 ms	**13 ms** (.34×)	153 ms	**50 ms** (.33×)	753 ms	**197 ms** (.26×)
$a = b$	39 ms	**15 ms** (.40×)	175 ms	**54 ms** (.31×)	723 ms	**205 ms** (.28×)
$a \ll b$	**1 ms**	12.9 ms (10.4×)	**5 ms**	50.8 ms (10.2×)	**20 ms**	201 ms (10.1×)
Advanced arithmetic						
a/b	**1.48 s**	*Not available*	**9.56 s**	*Not available*	**56.3 s**	*Not available*
$\sqrt{a}$	**1.41 s**		**9.88 s**		**57.8 s**	
Slots	64	2048	16	1024	4	512

Since $r = 2^{2n}/\hat{y}$, we have $1/r = \hat{y}/2^{2n}$, and since $\theta_j < 1$,

$$\frac{\theta_j}{r} = \theta_j \cdot \frac{\hat{y}}{2^{2n}} < \frac{\hat{y}}{2^{2n}},$$

hence $\varepsilon_{j+1} < \varepsilon_j^2 + \hat{y}/2^{2n}$. □

B.3 Lemma 5 (Accuracy after k steps)

Proof. We first introduce the two variables $c := \hat{y}/2^{2n}$ and $B_j := |\varepsilon_0|^{2^j} + 2c$. Observe that by Lemma 4 we have that $0 \le \varepsilon_{j+1} < \varepsilon_j^2 + c$. We want to show that this quadratic bound keeps verifying when j is increased. We define two cases: (i) the base case is such that $\varepsilon_1 < \varepsilon_0^2 + c \le |\varepsilon_0|^2 + 2c = B_1$. As for the (ii) inductive step, assume $\varepsilon_j \le B_j$. Then

$$\varepsilon_j^2 \le B_j^2 = |\varepsilon_0|^{2^{j+1}} + 4c\,|\varepsilon_0|^{2^j} + 4c^2.$$

Since $\ell \ge 4$, we have that

$$|\varepsilon_0|^{2^j} \le |\varepsilon_0| \le 2^{-\ell} \le 2^{-4},$$

so $4c\,|\varepsilon_0|^{2^j} \le c/4$. We do something similar and observe that, since

$$n \ge 8 \quad \text{then} \quad c < 2^{-n} \le 2^{-8},$$

then $4c^2 \leq c/64$. Hence $\varepsilon_j^2 < |\varepsilon_0|^{2^{j+1}} + c$, and

$$\varepsilon_{j+1} < \varepsilon_j^2 + c < |\varepsilon_0|^{2^{j+1}} + 2c = B_{j+1}.$$

Lastly, non-negativity of ε_j is inherited from Lemma 4. □

B.4 Theorem 2 (Quotient correctness)

Proof. To ease readability, let us define $Q := x/y$ and $\Delta := Q \cdot \varepsilon_k$. In particular Q is the "real" quotient, while Δ is simply Q scaled by ε_k. Since $\hat{y} = y \cdot 2^{n-\beta}$ we have that $r = 2^{2n}/\hat{y} = 2^{n+\beta}/y$, hence $c = 1/r = y/2^{n+\beta}$. Substituting $s_k = r(1 - \varepsilon_k)$,

$$x \cdot \frac{s_k}{2^{n+\beta}} = x \cdot \frac{r(1-\varepsilon_k)}{2^{n+\beta}} = \frac{x}{y}(1-\varepsilon_k) = Q - \Delta, \tag{12}$$

hence we can work directly on the difference $Q - \Delta$, which is precisely the output of the application of k Newton–Raphson iterations.

Step 1 (upper bound). We first define an upper bound: since $k \geq 1$, Lemma 4 gives $\varepsilon_k \geq 0$, so $\Delta \geq 0$ and by (12) $\hat{q} = \lfloor Q - \Delta \rfloor \leq \lfloor Q \rfloor$. Similarly as for one of Lemma 4 conclusions, we have that the estimate never becomes larger than the target.

Step 2 (reduction to $\Delta < 1$). As $\lfloor Q \rfloor - 1$ is an integer and $Q \geq \lfloor Q \rfloor$, the lower bound $\lfloor x \cdot (s_k/2^{n+\beta}) \rfloor \geq \lfloor Q \rfloor - 1$ follows if $Q - \Delta > \lfloor Q \rfloor - 1$. Therefore, to complete the proof, we are required to prove that $\Delta < 1$.

Step 3 (the case $\beta \geq 2$). By Corollary 4, the initial error is $|\varepsilon_0| < 2^{-\ell}$, so Lemma 5 applies and gives $\varepsilon_k \leq |\varepsilon_0|^{2^k} + 2\frac{\hat{y}}{2^{2n}}$. Multiplying by Q,

$$\Delta = Q\varepsilon_k \leq Q\,|\varepsilon_0|^{2^k} + 2Q\frac{\hat{y}}{2^{2n}}. \tag{13}$$

We bound the two summands. For the first, $2^k = n/\ell$ gives $|\varepsilon_0|^{2^k} < 2^{-\ell\, 2^k} = 2^{-n}$, while $x < 2^n$ and $y \geq 2^{\beta-1}$ give $Q < 2^{\,n-\beta+1}$; hence

$$Q|\varepsilon_0|^{2^k} < 2^{\,n-\beta+1} \cdot 2^{-n} = 2^{\,1-\beta}. \tag{14}$$

For the second, the factors of y cancel:

$$2Q\frac{\hat{y}}{2^{2n}} = 2 \cdot \frac{x}{y} \cdot \frac{\hat{y}}{2^{2n}} = 2 \cdot \frac{x}{y} \cdot \frac{y \cdot 2^{n-\beta}}{2^{2n}} = \frac{2x}{2^{\,n+\beta}} < 2^{\,1-\beta}, \tag{15}$$

again because $x < 2^n$. Substituting (14) and (15) into (13),

$$\Delta < 2^{1-\beta} + 2^{1-\beta} = 2^{\,2-\beta} \leq 1,$$

the last inequality by $\beta \geq 2$. The bound on Δ is strict, so $\Delta < 1$ as required.

Step 4 (the case $\beta = 1$). The previous approach works assuming $\beta \geq 2$ for the last step. In this case the bit length of the input is 1, i.e., the input is $y = 1$ since we assume $y > 0$. We now show that the initial hint is sufficient to recover the exact s_k. Set $\hat{y} = 2^{n-1}$ and $i = 0$ and $\sigma_0 = 2^{n-1} = \hat{y}$. Consequently, we have that $r = 2^{2n}/2^{n-1} = 2^{n+1}$ is an integer and the seed is exact:

$$s_0 = F(\sigma_0) = \left\lfloor 2^{2n}/2^{n-1} \right\rfloor = 2^{n+1} = r, \qquad \varepsilon_0 = 0.$$

By Lemma 4, $\Phi(s_j) = r(1 - \varepsilon_j^2)$, so $\varepsilon_j = 0$ implies $\Phi(s_j) = r \in \mathbb{Z}$ and hence $s_{j+1} = \lfloor \Phi(s_j) \rfloor = r$ and $\varepsilon_{j+1} = 0$. By induction $\varepsilon_k = 0$, so $\Delta = 0 < 1$, completing the proof. □

C Evaluation of square root

Consider this section as the analogous for Section 5, but instead of analyzing the division, we provide results for the evaluation of the square root function.

C.1 Correctness

We now repeat the analysis of Section 5 for $f(x) = 1/\sqrt{x}$, following an observation that we previously did: we approximate the reciprocal square root, whose Newton–Raphson iterations are division free, and recover $\sqrt{x}$ at the end. Remark that the LUT used for division computed:

$$F(\sigma_i) = \left\lfloor \frac{2^{2n}}{2^{n-1} + i\, 2^{n-1-\ell}} \right\rfloor.$$

which is precisely $F(\sigma_i) = \lfloor 2^{2n}/\sigma_i \rfloor$, with $\sigma_i = 2^{n-1} + i \cdot \delta$ for $0 \leq i < 2^\ell$. Now, we do something similar, namely computing an approximation of the function over uniformly spaced values. Let us define

$$F(\sigma_i) = \left\lfloor \frac{2^{3n/2}}{\sqrt{\sigma_i + \delta/2}} \right\rfloor, \tag{16}$$

Additionally, the division table is evaluated at the left endpoint $\sigma_i = 2^{n-1} + i \cdot \delta$, which makes the bucket ratio one-sided (9) and lets Corollary 4 exploit the sign. Note that, on the other hand, now we evaluate at the midpoint

$$\sigma_i + \tfrac{\delta}{2} = 2^{n-1} + (2i+1)2^{n-2-\ell},$$

which means that, since we will work with the normalized input $\hat{x} \in [2^{n-1}, 2^n)$, we have that $|\hat{x} - \sigma_i| \leq \delta/2$. Ultimately, since $\sigma_i > 2^{n-1}$,

$$\left| \frac{\hat{x}}{\sigma_i} - 1 \right| \leq \frac{\delta/2}{2^{n-1}} = 2^{-\ell-1}. \tag{17}$$

This two-sided bound replaces (9) in the square-root analysis. The table is precomputed in the clear, so either choice is free. Now we define

$$g(s) := \frac{1}{s^2} - \frac{\hat{x}}{2^{3n}},$$

that has its root at $r := 2^{3n/2}/\sqrt{\hat{x}} \in \left(2^n, \sqrt{2} \cdot 2^n\right]$. Since its derivative is $g'(s) = -2/s^3$, the update becomes

$$\Phi(s) = \left(s\left(3 \cdot 2^{3n} - \hat{x} \cdot s^2\right)\right)/2^{3n+1}. \tag{18}$$

However, in practice, we will compute the following update

$$u_j := \left\lfloor \frac{\hat{x} s_j^2}{2^{2n}} \right\rfloor, \quad s_{j+1} := \left\lfloor \frac{s_j(3 \cdot 2^n - u_j)}{2^{n+1}} \right\rfloor \tag{19}$$

As before we analyze the iterates $s_{j+1} := \lfloor \Phi(s_j) \rfloor$ through the j-th relative error

$$\varepsilon_j \;:=\; 1 - \frac{s_j}{r}, \qquad \text{given the } j\text{-th iterate } s_j = r(1-\varepsilon_j), \tag{20}$$

with ε_0 the error present in the initial seed $s_0 = F(\sigma_i)$ of Corollary 5.

Corollary 5 (Inverse square root seed accuracy). *Let $f(x) = 1/\sqrt{x}$. If $\ell \geq 4$ and $n \geq \ell + 4$, then*

$$|\varepsilon_0| < 2^{-\ell-1} < 2^{-\ell}.$$

Proof. We proceed similarly as in the proof of Corollary 4. Write $F(\sigma_i) = 2^{3n/2} f(\sigma_i) - \theta$ for some $\theta \in [0, 1)$. Then

$$\varepsilon_0 = 1 - \frac{2^{3n/2} f(\sigma_i) - \theta}{2^{3n/2} f(\hat{x})} = 1 - \frac{f(\sigma_i)}{f(\hat{x})} + \frac{\theta}{2^{3n/2} f(\hat{x})}.$$

We first bound the second term. Since $f(\hat{x}) = 1/\sqrt{\hat{x}}$ we have $t = \theta\sqrt{\hat{x}}/2^{3n/2}$, so from $\theta < 1$ and $\hat{x} < 2^n$ we have that, as before,

$$0 \;\leq\; t \;<\; 2^{n/2}/2^{3n/2} \;=\; 2^{-n}.$$

For the first term, $f(\sigma_i)/f(\hat{x}) = \sqrt{\hat{x}/\sigma_i}$. Set $u := \hat{x}/\sigma_i - 1$, so that we can bound $|u| \leq 2^{-\ell-1}$ by (17). Set $A = 1 - f(\sigma_i)/f(\hat{x})$ to be the first term of ε_0 and observe that,

$$A = 1 - \sqrt{1+u} = \frac{-u}{1+\sqrt{1+u}}.$$

Since $\sqrt{1+u} \geq 1 + u$ for $u \in [-1, 0]$ and $\sqrt{1+u} \geq 1$ for $u \geq 0$, in either case $1 + \sqrt{1+u} \geq 2 - 2^{-\ell-1}$, thus

$$|A| \leq \frac{2^{-\ell-1}}{2 - 2^{-\ell-1}} = \frac{1}{2^{\ell+2} - 1}.$$

Combining, $|\varepsilon_0| \le |A| + t < (2^{\ell+2}-1)^{-1} + 2^{-n}$. As $\ell \ge 4$ gives $(2^{\ell+2}-1)^{-1} \le 64/63 \cdot 2^{-\ell-2}$, and $n \ge \ell + 4$ gives $2^{-n} \le 2^{-\ell-4}$, we obtain

$$|\varepsilon_0| < (64/63 + 1)\, 2^{-\ell-4} < 2^{-\ell-1}.$$

□

Lemma 6 (One Newton–Raphson step, square-root case). *For every $j \ge 0$ and $\ell \ge 4$ and $|\varepsilon_j| < 2^{-\ell-1}$, there exist $\theta_j, \theta'_j \in [0,1)$ such that*

$$\varepsilon_{j+1} = \frac{1}{2}\varepsilon_j^2(3-\varepsilon_j) - \frac{(1-\varepsilon_j)\theta_j}{2^{n+1}} + \frac{\theta'_j}{r}.$$

Which gives

$$-\frac{1+2^{-\ell-1}}{2^{n+1}} \;<\; \varepsilon_{j+1} \;<\; 2\,\varepsilon_j^2 + 2^{-n}.$$

Proof. Let us consider the update in (19), and write $u_j = \hat{x} \cdot s_j^2 \cdot 2^{-2n} - \theta_j$ for $\theta_j \in [0,1)$. Then the update becomes

$$\begin{aligned} s_{j+1} &= \frac{s_j(3\cdot 2^n - u_j)}{2^{n+1}} - \theta'_j = \frac{s_j(3\cdot 2^n - \hat{x}\cdot s_j^2 \cdot 2^{-2n})}{2^{n+1}} + \frac{s_j\theta_j}{2^{n+1}} - \theta'_j \\ &= \Phi(s_j) + \frac{s_j\theta_j}{2^{n+1}} - \theta'_j \end{aligned}$$

by (18), for some $\theta'_j \in [0,1)$. Now, first observe that by using $\hat{x}\cdot 2^{-3n} = r^{-2}$ we have that

$$\Phi(s) = \left(s\left(3\cdot 2^{3n} - \hat{x}\cdot s^2\right)\right)/2^{3n+1} = \frac{s}{2}\left(3 - \frac{s^2}{r^2}\right),$$

which, chaining with $s_j = r(1-\varepsilon_j)$ (20), yields:

$$\begin{aligned} \Phi(s_j) &= \frac{r(1-\varepsilon_j)}{2}\left(3 - \frac{(r(1-\varepsilon_j))^2}{r^2}\right) = \frac{r(1-\varepsilon_j)}{2}(3-(1-\varepsilon_j)^2) \\ &= \frac{r(1-\varepsilon_j)}{2}(2-\varepsilon_j^2+2\varepsilon_j) = r\left(1 - \tfrac{1}{2}\varepsilon_j^2\,(3-\varepsilon_j)\right). \end{aligned}$$

This gives

$$s_{j+1} = r\left(1 - \tfrac{1}{2}\varepsilon_j^2(3-\varepsilon_j)\right) + \frac{s_j\,\theta_j}{2^{n+1}} - \theta'_j.$$

we can now divide by r and observe that $s_{j+1}/r = (1-\varepsilon_j)$, finally obtaining

$$\begin{aligned} 1-\varepsilon_{j+1} &= 1 - \tfrac{1}{2}\varepsilon_j^2(3-\varepsilon_j) + \frac{(1-\varepsilon_j)\theta_j}{2^{n+1}} - \frac{\theta'_j}{r} \\ \varepsilon_{j+1} &= \tfrac{1}{2}\varepsilon_j^2(3-\varepsilon_j) - \frac{(1-\varepsilon_j)\theta_j}{2^{n+1}} + \frac{\theta'_j}{r}, \end{aligned}$$

which gives the first claim of the lemma. To obtain the bound, we set t_1, t_2 and t_3 the three terms of the just obtained equation (i.e., $\varepsilon_{j+1} = t_1 - t_2 + t_3$). Now observe that, if $|\varepsilon_j| \leq 2^{-\ell-1}$, we have that

$$\begin{aligned} t_1 &= \tfrac{1}{2}\varepsilon_j^2(3-\varepsilon_j) = \varepsilon_j^2 \cdot (3-\varepsilon_j)/2 \leq 2\varepsilon_j^2 \quad \textit{Note that } (3-\varepsilon_j)/2 \leq 2 \\ t_2 &= \frac{(1-\varepsilon_j)\theta_j}{2^{n+1}} \quad \longrightarrow \quad 0 \leq t_2 < (1+2^{-\ell-1})/2^{n+1} \\ t_3 &= \theta'_j/r < \sqrt{\hat{x}}/2^{3n/2} < 2^{-n} \end{aligned}$$

Now, it is easy to show all terms are nonnegative, so the lower bound is given by t_2, on the other hand, the upper bound is given by $t_1 + t_3$. □

A couple of remarks: (i) the truncations again do not disturb the quadratic convergence: the relative error is squared (up to some factor) and the floors contribute at most one unit in the last place (2^{-n}). (ii) Unlike the division case, $\varepsilon_{j+1} \geq 0$ *does not* hold. An additional assumption will be required to make the final theorem work.

Lemma 7 (Accuracy after k steps, square-root case). *Let $\ell \geq 4$, $n \geq \ell+3$, and $|\varepsilon_0| \leq 2^{-\ell-1}$. Then for every $j \geq 1$*

$$-\frac{1+2^{-\ell-1}}{2^{n+1}} < \varepsilon_j < \tfrac{1}{2}\left(2|\varepsilon_0|\right)^{2^j} + 2^{1-n}.$$

Proof. By Lemma 6, whenever $|\varepsilon_j| \leq 2^{-\ell-1}$ we have

$$-\frac{1+2^{-\ell-1}}{2^{n+1}} \;<\; \varepsilon_{j+1} \;<\; 2\varepsilon_j^2 + 2^{-n}.$$

The idea is to iterate this bound: the seed error is squared at every step, while the rounding contributes less than 2^{-n} each time and does not accumulate beyond 2^{1-n}. Along the way we check that $|\varepsilon_j| \leq 2^{-\ell-1}$ keeps holding, so that Lemma 6 can be applied again. We proceed by induction on j.

Base case ($j = 1$). Since $|\varepsilon_0| \leq 2^{-\ell-1}$ by hypothesis, applying Lemma 6 with $j = 0$ gives the lower bound and

$$\varepsilon_1 < 2\varepsilon_0^2 + 2^{-n} = \tfrac{1}{2}\left(2|\varepsilon_0|\right)^2 + 2^{-n} < \tfrac{1}{2}\left(2|\varepsilon_0|\right)^2 + 2^{1-n}.$$

Inductive step. Assume both lower and upper bounds of ε_j of the statement hold for some $j \geq 1$. Since $2|\varepsilon_0| \leq 2^{-\ell}$ and $2^j \geq 2$, and since $n \geq \ell + 3$,

$$\tfrac{1}{2}\left(2|\varepsilon_0|\right)^{2^j} \leq 2^{-2\ell-1} \leq 2^{-\ell-2}, \qquad 2^{1-n} \leq 2^{-\ell-2},$$

so $\varepsilon_j \leq 2^{-\ell-1}$. On the other hand, $(1+2^{-\ell-1})/2^{n+1} < 2^{-n} \leq 2^{-\ell-3}$. Hence $|\varepsilon_j| \leq 2^{-\ell-1}$, and Lemma 6 combined with $\varepsilon_{j+1} < 2\varepsilon_j^2 + 2^{-n}$ gives the lower bound at $j+1$.

Step 1. If $\varepsilon_j \geq 0$, the upper bound applies directly. If $\varepsilon_j < 0$, the lower bound gives $|\varepsilon_j| < 2^{-n} < 2^{1-n}$, which is again below the upper bound. In both cases, $|\varepsilon_j| \leq \frac{1}{2}\left(2|\varepsilon_0|\right)^{2^j} + 2^{1-n}$, and squaring,

$$2\varepsilon_j^2 \leq \tfrac{1}{2}\left(2|\varepsilon_0|\right)^{2^{j+1}} + 4\left(2|\varepsilon_0|\right)^{2^j} 2^{-n} + 8 \cdot 2^{-2n}.$$

Step 2. Since $\left(2|\varepsilon_0|\right)^{2^j} \leq 2^{-2\ell} \leq 2^{-8}$, the middle term is at most $\frac{1}{64}\,2^{-n}$. Since $n \geq 7$, then $2^{-n} \leq 2^{-7}$, therefore the last term is at most $\frac{1}{16}\,2^{-n}$; their sum is below 2^{-n}.

By plugging *Step 1* and *2* into the recursion,

$$\varepsilon_{j+1} < 2\varepsilon_j^2 + 2^{-n} < \tfrac{1}{2}\left(2|\varepsilon_0|\right)^{2^{j+1}} + 2^{-n} + 2^{-n} = \tfrac{1}{2}\left(2|\varepsilon_0|\right)^{2^{j+1}} + 2^{1-n},$$

which is the upper bound at $j+1$. □

Note that the result of this lemma is not guaranteed for $j = 0$, where the only bound $|\varepsilon_0| \leq 2^{-\ell-1}$ is available. We now finally proceed in stating the main theorem to prove the correctness of the square root evaluation (which we cover in Algorithm 10).

Theorem 3 (Square root correctness). *Let $\ell \geq 4$ and $n \geq 4\ell$ be powers of two. Given $x \in [1, 2^n)$ and $p := \beta \bmod 2$, let $s_0 = F(\sigma_i)$ be the seed of Corollary 5. Let $s_1, \ldots, s_k$ with $k = \log(n/\ell) - 1$ be the iterates produced by k applications of (19). Then,*

$$\lfloor\sqrt{x}\rfloor - 1 \leq \left\lfloor \left\lfloor \frac{\hat{x} \cdot s_k}{2^n} \right\rfloor \cdot \frac{\lfloor 2^{\,n+p/2} \rfloor}{2^{\,2n-\lfloor\beta/2\rfloor}} \right\rfloor \leq \lfloor\sqrt{x}\rfloor.$$

Differently from the quotient, ε_k may be negative, so ρ is not an underestimate by construction. The proof shows that such overshoot is below 2^{-n-1} in relative terms, which is too small to push $\sqrt{x}$ past the next integer.

Proof. To ease readability, define

$$y := \left\lfloor \frac{\hat{x} \cdot s_k}{2^n} \right\rfloor, \qquad c_p := \lfloor 2^{\,n+p/2} \rfloor, \qquad \rho := \left\lfloor \frac{y \cdot c_p \cdot 2^{\lfloor\beta/2\rfloor}}{2^{2n}} \right\rfloor,$$

so that ρ is the middle term of the statement. Let $S := \sqrt{x}$, $a := \lfloor S \rfloor$ and $\lambda := \max(0, -\varepsilon_k)$. Since $r = 2^{3n/2}/\sqrt{\hat{x}}$ and $\hat{x} = x \cdot 2^{n-\beta}$, we have $\hat{x} \cdot r = S \cdot 2^{2n-\beta/2}$. Using $\lfloor\beta/2\rfloor + p/2 = \beta/2$ and substituting $s_k = r(1-\varepsilon_k)$,

$$\frac{\hat{x} \cdot s_k}{2^n} \cdot \frac{2^{\,n+p/2} \cdot 2^{\lfloor\beta/2\rfloor}}{2^{2n}} = \frac{S \cdot 2^{2n-\beta/2}(1-\varepsilon_k) \cdot 2^{\beta/2}}{2^{2n}} = S(1-\varepsilon_k), \tag{21}$$

namely, without the three floors, the algorithm would output exactly $S(1-\varepsilon_k)$.

Step 1 (error after k iterations). As $n \geq 4\ell$, we have $k \geq 1$ and $n \geq \ell + 4$, so Corollary 5 gives $|\varepsilon_0| < 2^{-\ell-1}$ and Lemma 7 applies. Since $2^k = n/(2\ell)$, we have $(2|\varepsilon_0|)^{2^k} < 2^{-\ell 2^k} = 2^{-n/2}$, hence

$$\varepsilon_k < 2^{-n/2-1} + 2^{1-n}. \tag{22}$$

On the other side, the lower bound of Lemma 6 is too weak for our purpose, so we sharpen it. Let $e := \varepsilon_{k-1}$, with $|e| \leq 2^{-\ell-1}$ by Corollary 5 ($k = 1$) or Lemma 7 ($k \geq 2$). Dropping the nonnegative term θ'_{k-1}/r in the identity of Lemma 6,

$$-\varepsilon_k \;\leq\; \frac{(1-e)\,\theta_{k-1}}{2^{n+1}} - \tfrac{1}{2}e^2(3-e) \;<\; (1-e)\,2^{-n-1} - \tfrac{1}{2}e^2(3-e).$$

If $e \geq 0$, the right-hand side is at most 2^{-n-1}. If $e < 0$, set $d := -e$: the right-hand side is at most $2^{-n-1} + d \cdot 2^{-n-1} - \frac{3}{2}d^2 \leq 2^{-n-1} + 2^{-2n-2}/6$, by maximizing over d. In both cases,

$$\lambda < 2^{-n-1}\left(1 + 2^{-n-3}\right). \tag{23}$$

Step 2 (upper bound). In this specific case, all the floors round down and all the factors are nonnegative, so by (21) we have $\rho \leq S(1-\varepsilon_k) \leq S(1+\lambda)$. Our goal now is to show that $\leq a$, we do so by contradiction.

Suppose $\rho \geq a+1$. Since $(a+1)^2$ is an integer larger than x, then $(a+1)^2 \geq x+1$, thus

$$x+1 \;\leq\; (a+1)^2 \;\leq\; \rho^2 \;\leq\; x(1+\lambda)^2 \quad\longrightarrow\quad 1 \;\leq\; x\,(2\lambda+\lambda^2).$$

However, by (23), $2\lambda+\lambda^2 < 2^{-n}(1+2^{-n-3}) + 2^{-2n-1} < 2^{-n}(1+2^{-n})$, and since $x \leq 2^n - 1$,

$$x\,(2\lambda+\lambda^2) < (1-2^{-n})(1+2^{-n}) < 1,$$

a contradiction. Hence $\rho \leq a$.

Step 3 (lower bound). Write $y = \hat{x} \cdot s_k/2^n - \theta$ and $c_p = 2^{\,n+p/2}(1-\eta)$, for some $\theta \in [0,1)$ and $\eta \in [0, 2^{-n-1/2})$, where $\eta = 0$ if $p = 0$. By (21), and since $c_p \cdot 2^{\lfloor \beta/2 \rfloor} \leq 2^{\,n+\beta/2}$,

$$\frac{y \cdot c_p \cdot 2^{\lfloor \beta/2 \rfloor}}{2^{2n}} = S(1-\varepsilon_k)(1-\eta) - \frac{\theta \cdot c_p \cdot 2^{\lfloor \beta/2 \rfloor}}{2^{2n}} \;\geq\; S - E,$$

where $E := S\varepsilon_k + S(1-\varepsilon_k)\,\eta + 2^{\beta/2-n}$. We bound the three summands of E using $S < 2^{\beta/2} \leq 2^{n/2}$. For the first, (22) gives $S\varepsilon_k < 1/2 + 2^{1-n/2}$. The second is nonzero only if β is odd, i.e., $\beta \leq n-1$: then $S < 2^{(n-1)/2}$, $1-\varepsilon_k \leq 1+\lambda < 2$ and $\eta < 2^{-n-1/2}$ give $S(1-\varepsilon_k)\,\eta < 2^{-n/2}$. The third is at most $2^{-n/2}$. Therefore,

$$E \;<\; \tfrac{1}{2} + 2^{\,2-n/2} \;\leq\; \tfrac{1}{2} + 2^{-6} \;<\; 1,$$

the second inequality by $n \geq 4\ell \geq 16$. Hence the argument of the floor defining ρ is larger than $S - 1 \geq a - 1$, and since $a-1$ is an integer, $\rho \geq a-1$. Notice that, unlike Theorem 2, no special case is needed for $\beta = 1$, as all the error terms scale with $S < 2^{\beta/2}$. □

The factor $\lfloor 2^{n+p/2} \rfloor$ in the statement un-do the normalization: since $\sqrt{x} = \sqrt{\hat{x}} \cdot 2^{(\beta-n)/2}$ and n is even, the exponent is not an integer when β is odd, and the missing $\sqrt{2}$ is provided by $\lfloor \sqrt{2} \cdot 2^n \rfloor$, whose rounding is the term η of Step 3. Hence, Theorem 3 already covers both parities of β, and we only miss turning its estimate into the exact result, which costs a single conditional increment.

Corollary 6 (Exact square root). *Under the assumptions of Theorem 3, let ρ be the middle term of its statement. Then,*

$$\lfloor \sqrt{x} \rfloor = \rho + \big((\rho+1)^2 \leq x\big),$$

Notice that $(\rho+1)^2 \leq x$ evaluates to 1 if it holds, and to 0 otherwise.

Proof. Let $a := \lfloor \sqrt{x} \rfloor$, so that $a^2 \leq x < (a+1)^2$. By Theorem 3, $\rho \in \{a-1, a\}$. If $\rho = a-1$, then $(\rho+1)^2 = a^2 \leq x$, so the increment is applied and the output is a. If $\rho = a$, then $(\rho+1)^2 = (a+1)^2 > x$, so the increment is not applied and the output is again a. □

To conclude, notice that the case of 16-bits fails to satisfy the condition $n \geq 4\ell$ for correctness, but it can be easily fixed by running forcing one Newton–Raphson iteration (i.e., pretending the number is 32-bit).

C.2 Evaluation, in practice

We first present, in Algorithm 10, the proposed algorithm to evaluate the square root, and then we do some discussion.

The main difference with Algorithm 9 is given by the observation about $n+\beta$ being odd or even. Given that n is a power of two, we focus on the bit length β. In particular, this "parity correction" arises because the exponent β cannot always be halved exactly using an integer shift. Writing $x = m \cdot 2^\beta$, the true square root is $\sqrt{x} = \sqrt{m} \cdot 2^{\beta/2}$. When β is even this poses no difficulty, since $2^{\beta/2}$ is itself an integer power of two and can be realized exactly by the shift $\ll \lfloor \beta/2 \rfloor$ (this is precisely the case then, in line 18, the corr factor becomes even_correction). When β is odd, however, $\beta/2$ is not an integer: writing $\beta = 2k+1$ gives $2^{\beta/2} = 2^k \cdot \sqrt{2}$, so the integer shift $\ll k$ recovers only 2^k and omits the residual factor of $\sqrt{2}$. Left uncorrected, this causes every result with an odd exponent to be too small by a factor of $\sqrt{2}$.

The bit parity $= \beta \bmod 2$ detects this case, and the correction term corr multiplies the mantissa result by $\sqrt{2}$ when parity $= 1$ and by 1 when parity $= 0$, so that $\sqrt{x} = \big(\sqrt{m} \cdot (\sqrt{2})^{parity}\big) \cdot 2^{\lfloor \beta/2 \rfloor}$ holds exactly regardless of whether β is even or odd.

D Noise analysis

The security notions of FHE schemes are unconventional, mainly because of the attack first presented in 2021 by Li and Micciancio [LM21] where they showed

Algorithm 10 Integer square root $\lfloor\sqrt{x}\rfloor$ in discrete–CKKS

Input: Word size $n \geq 4\ell$ being a power of two; LUT of size 2^ℓ; argument x of n bits, precomputed LUT $F(0), \ldots, F(2^\ell - 1)$ with $F(i)$ as in (16), $\boldsymbol{x} = \mathsf{Bits}(\mathrm{x}), x \in (0, 2^n)$
Output: $\lfloor\sqrt{x}\rfloor$

Procedure $\mathsf{Sqrt}(\boldsymbol{x}, n)$
  $\mathsf{odd_correction} \leftarrow \lfloor\sqrt{2} \cdot 2^n\rfloor$
  $\mathsf{even_correction} \leftarrow 2^n$
  $\beta \leftarrow$ bit length of x ▷ *Algorithm 7*
  parity $\leftarrow \beta \wedge 1$ ▷ *The parity of the bit length*
  $\hat{x} \leftarrow x \ll n - \beta$ ▷ *Algorithm 8*
Stage 1: LUT (seed)
  $i \leftarrow \hat{x} \gg (n - 1 - \ell) \bmod 2^\ell$ ▷ *Bucket index*
  $s \leftarrow F(i)$ ▷ *Algorithm 2*
Stage 2: Newton–Raphson refinement
  $k \leftarrow \log_2(n/\ell) - 1$ ▷ *Iterations needed for full $n/2$-bit accuracy*
  for $j \leftarrow 1$ **to** k
    Note: this is the Newton–Raphson update for $1/\sqrt{x}$
    $s_2 \leftarrow (s \cdot s \cdot \hat{x}) \gg 2n$
    $s \leftarrow (s \cdot (3 \cdot 2^n - s_2)) \gg (n + 1)$
Stage 3: recover the square root, paying attention to the parity of bit length
  $y \leftarrow (\hat{x} \cdot s) \gg n$
  corr $\leftarrow \mathsf{odd_correction} \cdot$ parity $+ \mathsf{even_correction} \cdot (1 -$ parity$)$
  result $\leftarrow (y \cdot$ corr$) \ll \lfloor\beta/2\rfloor$ ▷ *Variant of Algorithm 8*
  result $\leftarrow$ result $\gg 2n$
  result $\leftarrow$ result $+ ((\text{result} + 1) \cdot (\text{result}{+}1) \leq x)$ ▷ *Final correction*
  return result

that an adversary with access to a number of decryption errors of CKKS ciphertexts through a decryption oracle can mount passive key-recovery attack. Such attack is captured by the so-called IND-CPA^{D} model [LM21, Definition 2], and all countermeasures known to date require the user to compute a bound on the decryption error. Some time after, however, the attack has been generalized to also include exact-FHE schemes [CCP+24], including the scheme presented in this paper. The strongest solution up to date is to fix some set of parameters such that the probability failure of decryption is negligible, although this has, as a side effect, lower performance. We observe that our key-encapsulation method from [BTPH22] achieves a failure probability of 2^{-128} on the ModRaise step of the bootstrapping (Section E.1). Additionally, to ensure decryption correctness, one should bound the errors occurring during the evaluation of the homomorphic circuit. For this reason, we report some heuristically obtained precision bits on the output of the various integer operations available in our scheme in Table 10.

We stress, however, that these are purely heuristic results presented for illustrative purposes that should not be considered for formal security proofs. The main issue is that modeling such error is very hard and an active area of re-

Table 10: Heuristic noise analysis of our scheme. The reported values are precision bits (higher is better), computed as $-\lfloor \log_2(x) \rfloor$, where x is either the avg/min/max of the output CKKS error $\boldsymbol{e} \in \mathbb{R}^n$. The reported data is obtained over 1,000 independent executions

	Add/Comparison		Multiplication		Square root		Division	
Bits	Avg	(Min, Max)	Avg	(Min, Max)	Avg	(Min, Max)	Avg	(Min, Max)
16	19.6	(15.9, 32.3)	19.6	(15.2, 34.2)	19.6	(16.4, 32.7)	21.1	(16.6, 34.5)
32	19.5	(15.5, 32.8)	19.6	(16.3, 31.0)	19.6	(15.8, 31.5)	21.8	(16.6, 34.0)
64	19.4	(15.9, 33.9)	19.6	(15.1, 31.5)	19.6	(16.3, 31.8)	22.3	(17.3, 34.9)
128	19.6	(16.1, 32.7)	19.6	(16.2, 30.8)	19.6	(16.0, 31.1)	22.4	(17.6, 35.1)

search [BCM+25]. Hence, for simplicity, we can simply assume a threat model where the raw decryption result $p + e \in \mathbb{R}$ is not publicly shared, but only its rounded version $p \in \mathbb{Z}$ is, and leave the door open to further improvements of our scheme which can be instantiated with negligible probability of (decryption) failure.

E Deferred CKKS definitions and algorithms

We present in this section some CKKS related operations, including a high-level description of the (discrete) bootstrapping, reductions modulo 2, the Kogge–Stone adder (KSA) and the divide-and-conquer multiplication pseudo codes.

E.1 The bootstrapping operation

Observe that, due to how CKKS encoding works, after each multiplication, the scale of the resulting ciphertext is increased from Δ up to Δ^2, hence a so-called *rescaling* operation is required. This is simply implemented as a product by $1/\Delta$; by setting all the moduli in the chain as $q_i \approx \Delta$, we have that each multiplication *consumes* one moduli q_i in the chain $Q = \prod_i^L q_i$. After $L-1$ multiplications, the modulus becomes so small that no more rescaling operations can be executed, and in turn, multiplications. The bootstrapping operation in CKKS [CHK+18] raises the modulus back to larger values, and its core operation is the evaluation of (an approximation of) a modular reduction, since after the modulus raise some $q_0 I(x)$ factor is added to the message, with $I(x)$ being some polynomial whose coefficients heavily depend on the Hamming weight of the secret key. We briefly discuss its main steps in the following:

1. $\mathsf{SlotsToCoeffs}$ (StC): given a ciphertext encoding $[\,\mathsf{iDFT}(\Delta \boldsymbol{m})]_{q_0}$, homomorphically compute the DFT to obtain a ciphertext $\boldsymbol{c}$ encoding:

$$[\Delta \boldsymbol{m}]_{q_0}.$$

2. ModRaise: increase the modulus of the ciphertext from q_0 to Q. This causes the ciphertext to decrypt to the following:

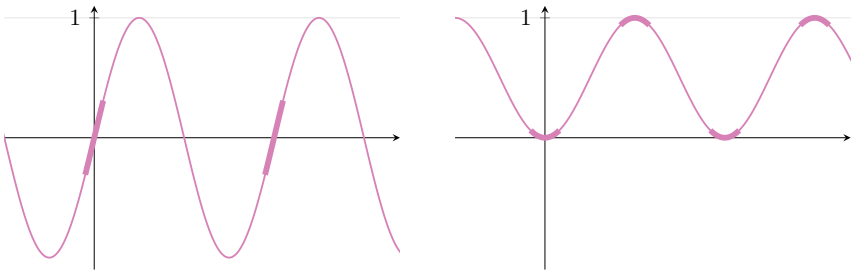
$$\Delta \boldsymbol{m} + q_0 I(x),$$

where $I(x)$ is a relatively small polynomial whose coefficients depend on the Hamming weight of the secret key.
3. CoeffsToSlots (CtS): re-encode the message (plus the new $q_0 I(x)$ term) back in the iDFT domain:

$$[\mathsf{iDFT}(\Delta \boldsymbol{m} + q_0 I(x))]_Q$$

4. EvalSin: since now the q_0 multiples are in the slots, it is possible to remove them by evaluating a $\sin(x)$-like function that simulates a modular reduction.

Towards functional bootstrapping. Recent advances in bootstrapping [BCKS24, AKP25] showed how it is possible to include in the evaluation of the bootstrapping the evaluation of some other function. In particular, we are interested in the *discrete bootstrapping* [BCKS24], whose goal is to both increase the modulus and reduce the amount of noise in discrete-CKKS ciphertexts. The main idea is as follows: instead of using a $\sin(x)$-like function to emulate the modular reduction, use a $\cos(x)$-like function assuming that all inputs are in the shape $\{0, 1\} + \varepsilon$ with $\varepsilon > 0$ relatively small. Now, given some proper scaling and shift of the $\cos(x)$ function, we end up with a bootstrapping function that, at the same time, removes the $q_0 I(x)$ term (since $\cos(x)$ is periodic) but also reduces the amount of noise ε [BCKS24, Lemma 1]. We report in Figure 9 an intuition on such procedure and we refer to [BCKS24] for more details about the used (discrete) bootstrapping.



(a) Standard bootstrapping for small $x \in \mathbb{R}$

(b) Binary bootstrapping for $x \in \{0, 1\}$

Fig. 9: Comparison of sin-like bootstrapping and cos-like bootstrapping (adapted from [BCKS24, Figure 1]). The highlighted area are where the inputs are expected to be (up to some scaling)

E.2 Modulo 2 operation

In CKKS, there are several ways to reduce the result of an addition modulo 2. The fastest one is to evaluate $a \oplus b = (a-b)^2$, although this has the problem of potentially increasing the error contained in both a and b [DMPS23, Lemma 2]. On the other hand, we also propose another polynomial that, at the cost of an additional multiplicative level, allows one to reduce modulo 2 and, at the same time, to apply a cleaning operation to the error. The polynomial is

$$h(x) := x^2(x-2)^2, \quad a \oplus b := h(a+b). \tag{24}$$

We present its plot in Figure 10. Also observe the error shrinks quadratically, as

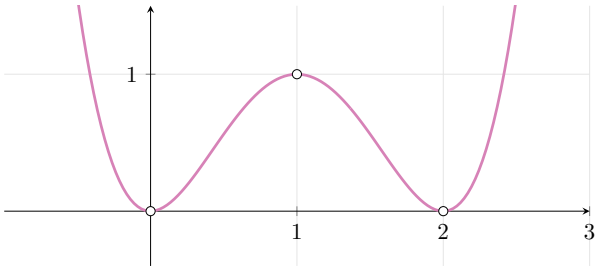


Fig. 10: The $x^2 \cdot (x-2)^2$ polynomial in the $(-1, 3)$ interval

proved in Lemma 8.

Lemma 8. *Let $h(x) = x^2(x-2)^2$. For each $x \in \{0, 1, 2\}$, there exist a constant $k > 0$ such that for all real e with $|e| < 0.1$, we have*

$$h(x+e) = h(x) + e',$$

where the new error e' satisfies

$$|e'| \leq ke^2.$$

Proof. Consider $x = 0$. We have

$$h(e) = e^2(e-2)^2$$

for $|e| < 0.1$. We can bound $e - 2$ as $|e-2| < 2.1$, leading to

$$|h(e)| < 4.41e^2$$

The same estimate holds for $x = 2$ by symmetry as $h(x) = h(2-x)$. Lastly, for $x = 1$, we compute

$$h(1+e) = (1-e^2)^2 = 1 + e^4 - 2e^2.$$

Thus the new error is $e' = e^4 - 2e^2$. Using the triangle inequality,

$$|e'| \leq |e|^4 + 2|e|^2.$$

Since $|e| < 0.1$, we have $|e|^2 < 0.01$ and $|e|^4 < 0.0001$, so

$$|e'| < 0.0001 + 0.02 < 3e^2.$$

□

E.3 Decoding algorithms for CKKS

The decoding for $\mathcal{R}$-CKKS is the standard CKKS one, thus simply the one described in Eq. (1). For $\mathcal{I}$-CKKS, we have that

$$\mathsf{Decode}_{\mathcal{I}}(\boldsymbol{e}) \leftarrow \lfloor \mathsf{Decode}(\boldsymbol{e}) \rceil \tag{25}$$

On the other hand, for $\mathcal{B}$-CKKS, to recover the n-bit integer value in radix-2, we have that

$$\mathsf{Decode}_{\mathcal{B}}(\boldsymbol{e}, n) \leftarrow \mathsf{Dec}\left(\lfloor \mathsf{Decode}(\boldsymbol{e}) \rceil, n\right) \tag{26}$$

Notice that, if $\boldsymbol{e}$ contains multiple (decomposed) integers, the decimal reconstruction Dec is applied independently over each integer. We avoid a pedantic formalization for readability. Of course a $\mathcal{B}$-CKKS can encode binary values that do not represent any integer, in such case one can simply use Eq. (25).

E.4 Kogge–Stone adder for CKKS

As previously mentioned, additions are performed using the Kogge–Stone adder (KSA) [KS73], described in Algorithm 3.

Initial propagate and generate. The first two vectors, $\boldsymbol{p}$ (propagate) and $\boldsymbol{g}$ (generate), are obtained respectively by computing $p_i = a_i \oplus b_i$ and $g_i = a_i \wedge b_i$. In our case, we can directly write $\boldsymbol{p} := \boldsymbol{a} + \boldsymbol{b} \bmod 2$ and $\boldsymbol{g} = \boldsymbol{a} \cdot \boldsymbol{b}$, which can be made to require only one multiplicative level. Refer to Section E.2 for details about the different approaches to evaluate the mod 2 operation in CKKS.

Computing prefix carries. This is the core and main loop of the KSA. Let us analyze the two lines 6-7 of Algorithm 3. Firstly, line 6 assigns, at g_i, the value

$$g_i := g_i + (p_i \wedge g_{i-2^d}). \tag{27}$$

The internal AND can be evaluated as $\boldsymbol{p} \cdot \mathrm{Rot}_{-2^d}(\boldsymbol{g})$, where $\mathsf{Rot}_i(\cdot)$ is the CKKS rotation by i positions to the left (negative values rotate to the right). On the other hand, the original KSA proposal required an external OR instead of a plain addition. However, we observe that such mod 2 is not required because the two terms are guaranteed never to be 1 at the same time. The reason is that if $g_i = 1$, it indicates a carry has already been generated from a lower-order bit that includes all the bits covered by $g_{i-2^{d/2}}$. Therefore, g_i and $p_i \wedge g_{i-2^d}$ can never overlap, making addition equivalent to OR for this line. We formalize in the following

Proposition 1. *In (27), the right-hand side never exceeds* 1*, so it coincides exactly with the Boolean update* $g_i \leftarrow g_i \vee (p_i \wedge g_j)$.

Proof. The sum $g_i + p_i g_j$ can exceed 1 only if $g_i = 1$ and $p_i g_j = 1$ simultaneously, i.e. only if $g_i = p_i = 1$ at the same index. But $p_i = 1$ means the resolved window represented by index i contains no generate bit (a pure propagate chain), which by definition of the generate signal forces $g_i = 0$ for that same window; conversely $g_i = 1$ arises only from an accumulated generate, which forces $p_i = 0$. Hence g_i and p_i are never simultaneously 1, so $g_i \cdot p_i \equiv 0$ at every index and stage, and $g_i + p_i g_j \in \{0, 1\}$ always equals $g_i \vee (p_i \wedge g_j)$.

Hence, we can evaluate the line simply as $\boldsymbol{g} := \boldsymbol{g} + (\boldsymbol{p} \cdot \mathsf{Rot}_{-2^d}(\boldsymbol{g}))$, consuming one multiplicative depth. Secondly, line 7 assigns, to p_i, the value of

$$p_i \wedge p_{i-2^d}.$$

The latter can easily be implemented as $\boldsymbol{p} := \boldsymbol{p} \cdot \mathsf{Rot}_{-2^d}(\boldsymbol{p})$. Notice that, for this to be true, we *require* our binary values to be followed by a zero. This is because the rotation is not a proper shift, so garbage values may be included when rotating. By setting a zero, this propagates through the wraparound, effectively masking most of the erroneous connections. The carries that come from the end of the array happen to be zero, so the result is still correct.

Proposition 2. *Let* $\boldsymbol{p}, \boldsymbol{g} \in \{0, 1\}^{n+1}$ *satisfy*

$$p_n = g_n = 0.$$

Then, at every stage of the Kogge–Stone prefix computation, replacing the left shift by a cyclic rotation, $\mathsf{Rot}_s(x)$*, produces the same result as a shift with zero fill, provided the updates are performed for all indices.*

Proof. Initially, the appended bit satisfies $p_n = g_n = 0$. The Kogge–Stone update is

$$g_i \leftarrow g_i \vee (p_i \wedge g_{i-s}), \qquad p_i \leftarrow p_i \wedge p_{i-s},$$

where indices with $i < s$ should receive a zero from the shifted operand. Using a cyclic rotation instead, these positions access the last entries of $\boldsymbol{p}$ and $\boldsymbol{g}$, which are zero by assumption. Hence every wrapped contribution vanishes and the rotation is equivalent to a zero-filled shift. Since the appended entries remain zero under the update, the argument holds inductively for every stage. □

Since the two lines do not depend on each other, each loop iteration consumes only one multiplicative depth.

Post processing. This is the final step and it involves computation of sum bits using OR operations. The original adder [KS73] required the following subroutine after the loop:

$$c_i \leftarrow g_{i-1} \vee (p_{i-1} \wedge c_{in}),$$

however notice that we assume not to take any carry in input[10]. So this assignment simply sets $c_i \leftarrow g_{i-1}$. Nonetheless, we are still required to set $c_0 \leftarrow 0$. Unfortunately, this would require us to mask all the positions but n, at the cost of one extra multiplication. We avoid this by simply assuming that the bits representation of the resulting value is followed by a zero, so we can simply rotate right by a position. This is similar to Proposition 2. This ensures that, when line 9 uses the value of g_0, it will use the 0 was manually forced. Notice that this is compatible with the multiplications logic that will require some zeros after each number, more details will be given later.

Proposition 3. *With no input carry, the following* $c_i \leftarrow g_{i-1} \vee (p_{i-1} \wedge c_{in})$ *simply becomes* $c_i \leftarrow g_{i-1}$. *It is possible to write* $g_0 \leftarrow 0$ *by simply appending a 0 after the* n *bits of the addends and by evaluating* $g \leftarrow \mathsf{Rot}_{-1}(g)$.

Proof. Since there is no input carry, we have $c_{\text{in}} = c_0 = 0$. Therefore, the carry recurrence

$$c_i \leftarrow g_{i-1} \vee (p_{i-1} \wedge c_{\text{in}})$$

reduces to

$$c_i = g_{i-1} \vee (p_{i-1} \wedge 0) = g_{i-1} \vee 0 = g_{i-1}.$$

Hence, the carry vector is

$$(c_0, c_1, \ldots, c_n) = (0, g_0, g_1, \ldots, g_{n-1}).$$

By assuming to have a zero next to the generate vector,

$$(g_0, g_1, \ldots, g_{n-1}, 0),$$

and applying a one-position right shift (denoted by Rot_1), we obtain exactly

$$(0, g_0, g_1, \ldots, g_{n-1}),$$

which is the required carry vector. □

Notice that the same argument remains valid when multiple values are packed into a single ciphertext. In this case, the zero appended after each value, when rotated, becomes the leading zero of the previous value.

Lastly, notice that since we mix AND and XOR operations, we can not use tricks such as lazy reduction à la [NHY+25].

E.5 Subtractions

As claimed before, subtractions are performed as $\boldsymbol{x} - \boldsymbol{y} = \boldsymbol{x} + \hat{\boldsymbol{y}} + 1$. We can thus use precisely the KSA adder, simply changing the first $\boldsymbol{g}$ computation by adding a 1 in the carry-in. We report a pseudocode of the subtraction based on KSA in Algorithm 11. The complexity of this procedure is precisely the same as the one of additions.

[10] The carry in can be useful if, e.g., we add two numbers plus an extra carry coming from a previous operation. This can happen if adding two $2n$-bit numbers using two n-bit adders. We observe that such modification can be done to our algorithm without major changes, as shown in the case of subtractions

Algorithm 11 Kogge–Stone subtraction

Input: Two binary numbers $\boldsymbol{x} = (x_0, \ldots, x_{n-1}, 0)$, $\boldsymbol{y} = (y_0, \ldots, y_{n-1}, 0)$ followed by one zero
Output: Subtraction $\boldsymbol{s} = (s_0 \ldots s_{n-1})$

Procedure $\mathsf{Sub}(\boldsymbol{x}, \boldsymbol{y})$
  $\hat{y}_i \leftarrow 1 - y_i$
  *Define the propagate (**p**) and generate (**g**) vectors:*
  $p_i \leftarrow x_i \oplus \hat{y}_i, \quad g_i \leftarrow x_i \wedge \hat{y}_i$
  Force the carry-in to be equal to 1
  $g_n = 1$
  Compute prefix carries using Kogge–Stone parallel prefix:
  for $d \leftarrow 0$ **to** $\log(n) - 1$
    $g_i \leftarrow g_i + (p_i \wedge g_{i-2^d})$
    $p_i \leftarrow p_i \wedge p_{i-2^d}$
  $c_i \leftarrow g_{i-1}, \quad c_0 \leftarrow 0$
  $s_i \leftarrow (x_i \oplus \hat{y}_i) \oplus c_i$ ▷ *For* $0 \leq i \leq n$
  return $\mathsf{BinBTS}(\boldsymbol{s})$

E.6 Divide-and-conquer multiplication for CKKS

We provide, in Algorithm 12, a version of the divide-and-conquer multiplication that makes only a single recursive call at each step, taking advantage of SIMD computations. Notice that we keep the algorithm description at a high-level, although we refer to the Python notebooks[11] available in the open-source repository for some plain implementations of it. Notice that, in practice, the maskings required in the first $\log(n)$ recursive calls can be done in a single processing when $\log(n) = 8$ to save some levels. This is the strategy that we followed in our code, refer to the two notebooks in the repository for the two different strategies.

On the role of Carry-save adder (CSA). In our divide-and-conquer multiplication, given the four multiplications:

$$\text{(i) } \boldsymbol{a}_\ell \cdot \boldsymbol{b}_\ell, \quad \text{(ii) } 2^{n/2} \cdot \boldsymbol{a}_\ell \cdot \boldsymbol{b}_h, \quad \text{(iii) } 2^{n/2} \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_\ell, \quad \text{(iv) } 2^n \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_h,$$

we are required to add them – with carry – to obtain the final result. Notice that the multiplications by powers of two can be implemented using shifts (Section 4), although we would need to evaluate at least two additions with carry to obtain the final addition result. Instead, we use the carry-save adder (CSA), that, given $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$, returns $\boldsymbol{c}', \boldsymbol{s}$ such that $\boldsymbol{c}' + \boldsymbol{s} = \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$. Since our operands are four, we can evaluate two composed CSA (which are less expensive than an addition with carry) to produce two operands that, once added with KSA, give as output the result of the final addition. We start by providing a definition of the CSA in Algorithm 13. Notice that the last polynomial $p(x) := -\frac{1}{3}x^3 + \frac{3}{2}x^2 - \frac{7}{6}x$ returns 0 in positions in which the sum is either 0 or 1, and returns 1 otherwise (sum equal to 2 or 3). We provide a plot of $p(x)$ for convenience in Figure 11.

[11] github.com/lorenzorovida/advanced-arithmetic-fhe/notebooks

Algorithm 12 Divide-and-conquer n-bit multiplication

Input: Two n-bit numbers $\boldsymbol{a} = (a_0, \dots, a_{n-1})$, $\boldsymbol{b} = (b_0, \dots, b_{n-1})$
Output: Product $\boldsymbol{m} = (m_0 \dots m_{2n-1})$

Procedure Multiplication$(\boldsymbol{a}, \boldsymbol{b}, n)$
if $n = 4$ **then**
  All the base cases are computed in one operation in parallel
  $\boldsymbol{r} := \mathcal{I}\text{-to-}\mathcal{B}(\mathcal{B}\text{-to-}\mathcal{I}(\boldsymbol{a}) \cdot \mathcal{B}\text{-to-}\mathcal{I}(\boldsymbol{b}))$
  return $3\boldsymbol{r}^2 - 2\boldsymbol{r}^3$ ▷ *Error-reducing polynomial* [DMPS23]
else
  $\boldsymbol{m}_\ell :=$ mask for low bits, contains ones in $[0, n/2)$
  $\boldsymbol{m}_h :=$ mask for high bits, contains ones in $[n/2, n)$
  $\boldsymbol{x}' := \boldsymbol{x} \cdot \boldsymbol{m}_\ell + \mathrm{Rot}_{-n/2}(\boldsymbol{x} \cdot \boldsymbol{m}_\ell) + \mathrm{Rot}_{-n^2-n}(\boldsymbol{x} \cdot \boldsymbol{m}_h) + \mathrm{Rot}_{-n^2}(\boldsymbol{x} \cdot \boldsymbol{m}_h)$
  $\boldsymbol{y}' := \mathrm{Rot}_{n/2}(\boldsymbol{y} \cdot \boldsymbol{m}_\ell) + \mathrm{Rot}_{-n}(\boldsymbol{y} \cdot \boldsymbol{m}_h) + \mathrm{Rot}_{-n/2}(\boldsymbol{y} \cdot \boldsymbol{m}_\ell) + \mathrm{Rot}_{-n^2-n}(\boldsymbol{y} \cdot \boldsymbol{m}_h)$
  Now we have $\boldsymbol{x}' = (\boldsymbol{x}_\ell, 0, \boldsymbol{x}_\ell, 0, \boldsymbol{x}_h, 0, \boldsymbol{x}_h, 0)$ *and* $\boldsymbol{y}' = (\boldsymbol{y}_\ell, 0, \boldsymbol{y}_h, 0, \boldsymbol{y}_\ell, 0, \boldsymbol{y}_h, 0)$. *The size of zeros is* $n/2$ *and each will be filled in the next recursive calls*
  $\boldsymbol{m} :=$ Multiplication$(\boldsymbol{x}', \boldsymbol{y}', n/2)$
  The partial results are shifted and added with CSA. The Mask *function masks the values of interest, depending on the term*
  $\boldsymbol{t}_1 :=$ Mask$(\boldsymbol{m})$ ▷ *Recover* $\boldsymbol{a}_\ell \cdot \boldsymbol{b}_\ell$
  $\boldsymbol{t}_2 := \mathrm{Rot}_{-n/2}(\text{Mask}(\boldsymbol{m}))$ ▷ *Recover* $2^{n/2} \cdot \boldsymbol{a}_\ell \cdot \boldsymbol{b}_h$
  $\boldsymbol{t}_3 := \mathrm{Rot}_{-n/2}(\text{Mask}(\boldsymbol{m}))$ ▷ *Recover* $2^{n/2} \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_\ell$
  $\boldsymbol{t}_4 := \mathrm{Rot}_{-n}(\text{Mask}(\boldsymbol{m}))$ ▷ *Recover* $2^n \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_h$
  return DoubleCSA$(\boldsymbol{t}_1, \boldsymbol{t}_2, \boldsymbol{t}_3, \boldsymbol{t}_4)$ ▷ Refer to Algorithm 14.
end if

Next, we can use this as a building block for our four operands adder. We present the final procedure in Algorithm 14. To conclude, observe that the multiplicative cost of two KSA is equal to

$$2(2 + \log(n)), \tag{28}$$

by Algorithm 3, while the cost of two CSA, followed by a KSA, is

$$4 + 2 + \log(n). \tag{29}$$

In general, we have that performing two KSA is more convenient when Eq. (28) is smaller than Eq. (29), that is when

$$2(2 + \log(n)) < 4 + 2 + \log(n).$$

This holds as long as $0 < n < 4$, and for our cases we are always outside of that interval. Hence, it is always convenient to use two CSA and one KSA.

Algorithm 13 Carry-save adder (CSA)

Input: Three n-bit numbers $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$
Output: Sum $\boldsymbol{s}$ and carry $\boldsymbol{c}$ such that $\boldsymbol{s} + \boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$
Procedure $\mathsf{CSA}(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c})$
  Compute the first XOR:
  $\boldsymbol{s} := (\boldsymbol{a} - \boldsymbol{b})^2$
  Compute the second XOR:
  $\boldsymbol{s} := (\boldsymbol{s} - \boldsymbol{c})^2$
  Compute the majority bits:
  $\boldsymbol{t} := \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$
  $\boldsymbol{c} := -\frac{1}{3}\boldsymbol{t}^3 + \frac{3}{2}\boldsymbol{t}^2 - \frac{7}{6}\boldsymbol{t}$
  We rotate by one the carry to move it in the correct position:
  return $\boldsymbol{s}, \mathrm{Rot}_{-1}(\boldsymbol{c})$

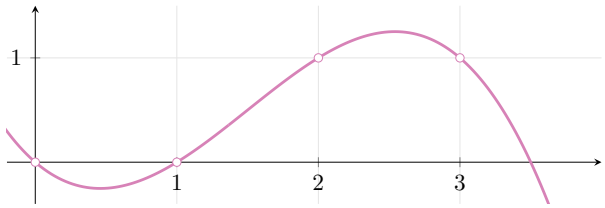


Fig. 11: The $-\frac{1}{3}x^3 + \frac{3}{2}x^2 - \frac{7}{6}x$ polynomial in the $(0, 4)$ interval

F LUT seed examples

F.1 Integer division with $n = 8$, $\ell = 4$

To give a practical intuition for the LUT construction applied to integer division, we consider the same small, fully enumerable case: $n = 8$ (an 8-bit denominator) and $\ell = 4$ (a 16 entry LUT). The goal of our example is to compute

$$q = \left\lfloor \frac{a}{b} \right\rfloor \qquad \text{with } a = 200, b = 33$$

using a fixed-point reciprocal approximation. Instead of storing the real function $1/x$, the LUT stores the scaled integer reciprocal

$$F(x) = \left\lfloor \frac{2^{2n}}{x} \right\rfloor, \quad \text{with quotient } \hat{q} = \left\lfloor \frac{a \cdot F(b)}{2^{2n}} \right\rfloor.$$

Building the LUT. The normalized denominator range is $[2^{n-1}, 2^n) = [2^7, 2^8) = [128, 256)$, which has width 128. With $\ell = 4$, the range is divided into $2^\ell = 16$ buckets, each of width $\delta = 2^{n-1-\ell} = 2^{7-4} = 8$. Bucket i therefore covers

$$[\sigma_i, \sigma_i + 8)$$

with left endpoint

$$\sigma_i = 128 + 8i, \qquad i \in [0, 16).$$

Algorithm 14 Reducing $a + b + c + d$ to a single KSA

Input: Four n-bit numbers $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d} \in \{0,1\}^n$
Output: Sum $\boldsymbol{s} = (s_0 \dots s_{n-1})$ and carry-out s_n
1: *Compute the first CSA:*
2: $\boldsymbol{s}_1, \boldsymbol{c}_1 := \mathsf{CSA}(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c})$
3: *Compute the second CSA:*
4: $\boldsymbol{s}_2, \boldsymbol{c}_2 := \mathsf{CSA}(\boldsymbol{s}_1, \boldsymbol{c}_1, \boldsymbol{d})$
5: *Return the actual addition with carry:*
6: **return** $\mathsf{KSA}(\boldsymbol{s}_2, \boldsymbol{c}_2)$

We choose a reciprocal precision of $2n = 16$. The LUT stores the scaled reciprocal evaluated at the bucket left endpoints:

$$F(\sigma_i) = \left\lfloor \frac{2^{2n}}{\sigma_i} \right\rfloor = \left\lfloor \frac{2^{16}}{\sigma_i} \right\rfloor .$$

We report in Table 11 all the entries.

Table 11: The reciprocal lookup table for $n = 8$, $\ell = 4$, and $2n = 16$. Each entry $\lfloor 2^{16}/\sigma_i \rfloor$ is computed once, offline, and stored as an integer.

i	σ_i	$2^{16}/\sigma_i$	$F(\sigma_i)$	i	σ_i	$2^{16}/\sigma_i$	$F(\sigma_i)$
0	128	512.000	512	8	192	341.333	341
1	136	481.882	481	9	200	327.680	327
2	144	455.111	455	10	208	315.077	315
3	152	431.158	431	11	216	303.407	303
4	160	409.600	409	12	224	292.571	292
5	168	390.095	390	13	232	282.483	282
6	176	372.364	372	14	240	273.067	273
7	184	356.174	356	15	248	264.258	264

Evaluating the LUT. Now we can compute the LUT (online phase). In our example we want to compute the division a/b with $a = 200$ and $b = 33$. Recall that the bucket index i is computed as

$$i = \lfloor (\hat{b} - 2^{n-1})/\delta \rfloor$$

with $\delta = 2^{n-1-\ell}$. We first compute $\hat{b} \leftarrow b \cdot 2^{n-\beta}$, and notice that the bit length β of b is $\beta = 6$ (i.e., 6 bits are required to represent $b = 33$). This gives us

$\hat{b} = 33 \cdot 2^{8-6} = 33 \cdot 4 = 132$. We then compute the index:

$$i = \left\lfloor \frac{\hat{b} - 2^{n-1}}{\delta} \right\rfloor = \left\lfloor \frac{132 - 128}{8} \right\rfloor = 0.$$

Therefore, the LUT returns the reciprocal seed

$$x_0 = F(\sigma_0) = 512.$$

The exact value of $2^{2n}/\hat{b}$ is $F(132) = \lfloor 2^{16}/132 \rfloor = 496$. Hence the seed error is

$$|F(132) - F(\sigma_0)| = |496 - 512| = 16. \quad \text{with rel. err.} = \frac{16}{496} = 0.03225 \approx 2^{-5}.$$

This is consistent with the expected LUT accuracy. With $\ell = 4$ address bits, the first-order interpolation error is $O(2^{-\ell}) = O(2^{-4})$. The quotient estimate obtained from the LUT seed is

$$q = \left\lfloor \frac{200 \cdot 512}{2^{n+\beta}} \right\rfloor = \left\lfloor \frac{200 \cdot 512}{2^{14}} \right\rfloor = \lfloor 6.25 \rfloor = 6.$$

The exact integer quotient is $\lfloor \frac{200}{33} \rfloor = 6$, thus the LUT seed is already correct and no Newton–Raphson iterations are even required.

F.2 Square root with $n = 8$, $\ell = 4$

We now provide another example, this time using the $f(x) = \sqrt{x}$ function. Notice that such example is purely didactic, in practice to compute $\sqrt{x}$ we approximate its reciprocal as the Newton–Raphson iterations of $\sqrt{x}$ require the computation of a division.

Building the LUT. The normalized range is

$$[2^{n-1}, 2^n) = [2^7, 2^8) = [128, 256),$$

of width 128. With $\ell = 4$, there are $2^\ell = 16$ buckets, each of width

$$\delta = 2^{n-1-\ell} = 2^{7-4} = 2^3 = 8.$$

Bucket i therefore covers $[\sigma_i, \sigma_i + 8)$ with left endpoint $\sigma_i = 128 + 8i$, for $i \in [0, 16)$. Choosing the scaling exponent $2n = 16$, the scaled target is

$$F(x) = \lfloor 2^8 \sqrt{x} \rfloor,$$

and the table stores $F(\sigma_i)$ for each bucket. Table 12 gives every entry.

Table 12: The full lookup table for $n = 8$, $\ell = 4$, $f(x) = \sqrt{x}$. Each $F(\sigma_i) = \lfloor 2^8 \sqrt{\sigma_i} \rfloor$ is computed once, offline, and stored exactly.

i	σ_i	$2^8\sqrt{\sigma_i}$	$F(\sigma_i)$	i	σ_i	$2^8\sqrt{\sigma_i}$	$F(\sigma_i)$
0	128	2896.31	2896	8	192	3547.24	3547
1	136	2985.45	2985	9	200	3620.38	3620
2	144	3072.00	3072	10	208	3692.08	3692
3	152	3156.18	3156	11	216	3762.41	3762
4	160	3238.14	3238	12	224	3831.46	3831
5	168	3318.14	3318	13	232	3899.27	3899
6	176	3396.22	3396	14	240	3965.93	3965
7	184	3472.55	3472	15	248	4031.49	4031

Evaluating the LUT. Let $x = 150$, we thus want to compute an estimate of $\sqrt{x} = \sqrt{150} = 12$ (assuming integer square root, i.e. $\lfloor \sqrt{x} \rfloor$). We compute $\hat{x} = x \cdot 2^{n-\beta}$, and given bit length of x equal to $\beta = 8$, we have that

$$\hat{x} = 1\hat{5}0 = 150 \cdot 2^{8-8} = 150$$

Its bucket index is: $i = \lfloor (150 - 128)/8 \rfloor = 2$, hence $\sigma_2 = 144$ according to Table 12. Therefore, the LUT returns the seed $s = F(\sigma_2) = 3072$. The true scaled value at $x = 150$ is

$$F(150) = \left\lfloor \sqrt{150} \cdot 256 \right\rceil = 3135,$$

so the seed's error is

$$|F(\sigma_2) - F(x)| = |3072 - 3135| = 63, \qquad \text{relative error} = \frac{63}{3135} \approx 2^{-5.6}.$$

This is consistent with (and slightly better than) the $O(2^{-\ell}) = O(2^{-4})$ bound of Corollary 5: $x = 150$ happens to sit close to σ_2, so the linear term in the Taylor expansion is small. In particular, to recover the final values simply shift the result:

$$F(150) \gg \beta = 3135 \gg 8 = 12$$

Which is correct as $\lfloor \sqrt{150} \rfloor = 12$.

G Comparing LUT evaluations

In this section we compare our approach to LUTs with respect to the current literature. We claim that our LUT evaluation are faster with respect to other works, at the cost of having a reduced throughput.

Comparison with [AKP25]. Our method of using polynomials to perform interpolations have many common points with the nice paper by Alexandru, Kim and Polyakov [AKP25], but the usage and execution are fairly different. First of all, their approach is to use Hermite interpolations with vanishing derivatives. This has the advantage of "cleaning" the discrete-CKKS error while evaluating a LUT. On the other hand, our $\mathcal{I}$-to-$\mathcal{B}$ conversion uses an approximation of a trigonometric polynomial that does not have any constraint over the derivatives. This has the advantage of being faster to evaluate, but requires a subsequent cleaning step — that, in our case, we do during the bootstrapping [BCKS24]. While in the first place it might be counterintuitive, our approach requires more levels to evaluate LUT + cleaning, but has, as a consequence, a faster final execution and the freedom not to rely on bootstrapping to evaluate the LUT.

Comparison with [DAP+26]. The main advantage of the large-LUT functional bootstrapping construction of Dumezy *et al.* [DAP+26] over the general functional bootstrapping (AKP-FBT) of [AKP25] is its significantly better scalability with the LUT size. In AKP-FBT, an arbitrary LUT $f : \mathbb{Z}_P \to \mathbb{Z}_P$ is represented by a trigonometric Hermite interpolation polynomial whose degree grows essentially as $P = 2^b$, where b is the number of input bits. Consequently, the cost of homomorphically evaluating the interpolation polynomial grows rapidly as the LUT becomes larger, making AKP-FBT increasingly expensive for large LUTs. In [DAP+26], they avoid this large-degree polynomial by decomposing the input into smaller digits and evaluating the LUT through a multiplexer tree. Their Binary Multiplexer Tree (BMT-FBT) and Collapsed Multiplexer Tree (CMT-FBT) constructions combine functional digit decomposition with small LUT evaluations, allowing the large LUT to be expressed as a composition of much smaller functions.

By choosing an appropriate digit size, CMT-FBT substantially reduces the multiplicative depth while avoiding the exponential-in-b polynomial degree of our work, but also in AKP-FBT. A key ingredient in the construction is the decomposition of the LUT input into a collection of smaller digits (or, in the binary case, individual bits). This decomposition is essential because the multiplexer-tree representation relies on selecting between smaller sub-LUTs according to the value of each digit. The LUT can then be recursively expressed through binary selection operations of the form $\mathsf{MUX}(x_i, f_0, f_1)$, where f_0 and f_1 are the two sub-LUTs corresponding to $x_i = 0$ and $x_i = 1$, respectively. This transforms the evaluation of one large LUT into a tree of conditional statements, whose depth is determined by the number of decomposition levels rather than by the size of the original LUT.

Note that input decomposition is fundamental to the scalability of the approach, while binary decomposition is best viewed as the basic construction from which the more general digit-based and collapsed-tree variants are derived. For this reason we believe that our Algorithm 1 can be beneficial for CMT-FBT constructions, also observing that our binary decomposition can easily be gener-

alized to a digit decomposition for radix $r \leq 8$ by using the following definition.

Definition 4 (generalizes Definition 3). *Let $x \in \mathbb{Z}_{r^b}$, where $r \geq 2$ and $0 \leq i < b$. The i-th digit of x in radix r is given by*

$$e_i^{(r)}(x) := \lfloor x/r^i \rfloor \bmod r.$$

We see as a potential future work a CMT-FBT-like construction based on some of our ideas.

Comparison with [DS26]. In a very recent paper [DS26], Dumezy and Suvanto present a very elegant way to deal with LUTs in CKKS. In particular, they encode values in what they call *block root-of-unity encoding* (BRU). In practice, Instead of storing m directly, they store the character evaluations of m. For the BRU encoding over $(\mathbb{Z}_t, +)$, the value m is represented as the vector of roots of unity:

$$\mathrm{BRU}_t(m) = \left(\zeta_t^m, \zeta_t^{2m}, \ldots, \zeta_t^{(t-1)m}\right).$$

In particular the characters of the group are given by $\chi_k(m) = \zeta_t^{km}$. The key observation is that the characters span all functions on the alphabet. Any function $f : \mathbb{Z}_t \to \mathbb{C}$ has a unique expansion:

$$f(m) = \hat{f}(0) + \sum_{k=1}^{t-1} \hat{f}(k)\, \zeta_t^{km} = \hat{f}(0) + \sum_{k=1}^{t-1} \hat{f}(k)\, \chi_k(m).$$

so evaluating f is just a weighted sum of the slots with precomputed plaintext weights $\hat{f}(k)$. That is a precisely a plaintext-ciphertext linear map in the form $\omega \mapsto A\omega + b$ which costs exactly one multiplicative level (independently from t). However, one strong assumption is on the BRU encoding which, if available during encryption time, allows to evaluate LUT in (possibly) the fastest way possible. On the other hand, if values come from another computation or need to be converted into BRU form, such transformation may take some seconds and thus the cost of the LUT is translated into the cost of such transformation [DS26, Table 12]. On the other hand, our LUT evaluation takes more multiplications but, overall, is still faster than the Standard $\to$ BRU evaluation according to [DS26, Table 12], under the same parameters set. Their methods remain preferable if the use case requires the evaluation of many LUTs.

H Real to integers, and how to adapt CKKS neural networks to the hard-label setting

We mentioned that one can transform real values to integers via the Argmax function, which is a typical operation performed at the end of neural network inference, although this step is typically skipped in encrypted implementations

because of its impracticability. We however observe that recent works, mainly [MEHP25,KYML25,RLB26], proposed very efficient procedures to compute such function in the context of sorting real numbers in CKKS. In general, we observe that such reals to integer transformation is computed by many functions: Floor, Ceil, Round, Rank, Threshold, and so on. We therefore believe that, within a framework that handles at the same time computations over $\mathbb{R}$ and $\mathbb{Z}$, having access to such functions is very important. While most of them are out of the scope of the paper, we sketch in Algorithm 15 the example of the evaluation of the Argmax function, extended from [RLB26, Algorithm 1]. Refer to [RLB26, Definition 1, 2] for a definition of $\boldsymbol{c}_{\text{rep}}$ and $\boldsymbol{c}_{\text{exp}}$. Optimizations from [RLB26, Section 3.1.2] show also how to handle repeated elements.

Algorithm 15 Argmax in CKKS, sketch

Input: Two encodings $\boldsymbol{c}_{\text{rep}}, \boldsymbol{c}_{\text{exp}} \in \mathbb{R}^{n^2}$ of the input vector $\boldsymbol{c} \in \mathbb{R}^n$ with no elements at distance at most $\delta > 0$, a polynomial approximation $p(x)$ of the function $\text{sgn}(x)$ with desired accuracy in $[-1, -\delta] \cup [\delta, 1]$ and a polynomial approximation $s(x)$ of $\text{sinc}(x)$ in $[-n, 0]$

Output: The index of the maximum element in $\boldsymbol{c}$ (i.e., the Argmax)

Procedure Argmax($\boldsymbol{c}_{\text{rep}}, \boldsymbol{c}_{\text{exp}}$)
  $\boldsymbol{c}_\Delta \leftarrow \text{Sub}(\boldsymbol{c}_{\text{rep}}, \boldsymbol{c}_{\text{exp}})$
  $\boldsymbol{s} \leftarrow p(\boldsymbol{c}_\Delta)$
  for $i := 0$ to $\log(n)$
    $\boldsymbol{s} \leftarrow \text{Add}(\boldsymbol{s}, \text{Rot}_{n \cdot 2^i}(\boldsymbol{s}))$ ▷ *Rotate and sum*
  $\boldsymbol{idx} \leftarrow \text{Add}(\boldsymbol{s}, \boldsymbol{0.5}_{n^2})$
  The following returns 1 only in the slot containing the maximum index
  $\boldsymbol{m} \leftarrow \text{sinc}(\boldsymbol{idx} - (n-1))$
  Below all values will be zeros, except the index of the maximum element
  $\boldsymbol{m} \leftarrow \boldsymbol{m} \cdot (1, 2, \ldots, n)$
  return $\boldsymbol{m}$

Notice that the complexity of this algorithm, under the parameters in Table 4 can be 2 bootstrappings, as the main bottleneck is represented by the evaluation of the approximations of the $\text{sinc}(x)$ and $\text{sgn}(x)$ functions.

H.1 Defending encrypted NNs from model extraction

In 2020, Carlini, Jagielski and Mironov showed [CJM20] that the problem of *model extraction* is actually a cryptanalytic problem. Since then, more works [CCSH+25, LSE+26] improved the attacks.

It is well-known that the *hard-label* setup is the hardest to attack [IMT25], as one only has access to the output class of the network. On the other hand, attacks are much more successful if one has access to the output probabilities (or logits). The question then becomes: how to convert the output logits of a NN to the index of a class? The simple answer lies in the previously defined function Argmax. Indeed, it is possible to apply Algorithm 15 to the output logits of a

NN to derive the corresponding class. This costs 2 bootstrappings, but strongly increases the security of the encrypted model against model extraction attacks.